

Evaluating SZZ Implementations: An Empirical Study on the Linux Kernel

Yunbo Lyu , Hong Jin Kang , Ratnadira Widyasari , *Graduate Student Member, IEEE*, Julia Lawall , and David Lo , *Fellow, IEEE*

Abstract—The SZZ algorithm is used to connect bug-fixing commits to the earlier commits that introduced bugs. This algorithm has many applications and many variants have been devised. However, there are some types of commits that cannot be traced by the SZZ algorithm, referred to as “ghost commits”. The evaluation of how these ghost commits impact the SZZ implementations remains limited. Moreover, these implementations have been evaluated on datasets created by software engineering researchers from information in bug trackers and version controlled histories. Since Oct 2013, the Linux kernel developers have started labelling bug-fixing patches with the commit identifiers of the corresponding bug-inducing commit(s) as a standard practice. As of v6.1-rc5, 76,046 pairs of bug-fixing patches and bug-inducing commits are available. This provides a unique opportunity to evaluate the SZZ algorithm on a large dataset that has been created and reviewed by project developers, entirely independently of the biases of software engineering researchers. In this paper, we apply six SZZ implementations to 76,046 pairs of bug-fixing patches and bug-introducing commits from the Linux kernel. Our findings reveal that SZZ algorithms experience a more significant decline in recall on our dataset ($\downarrow 13.8\%$) as compared to prior findings reported by Rosa et al., and the disparities between the individual SZZ algorithms diminish. Moreover, we find that 17.47% of bug-fixing commits are ghost commits. Finally, we propose Tracing-Commit SZZ (TC-SZZ), that traces all commits in the change history of lines modified or deleted in bug-fixing commits. Applying TC-SZZ to all failure cases, excluding ghost commits, we found that TC-SZZ could identify 17.7% of them. Our further analysis based on *git log* found that 34.6% of bug-inducing commits were in the function history, 27.5% in the file history (but not in the function history), and 37.9% not in the file history. We further evaluated the effectiveness of ChatGPT in boosting the SZZ algorithm’s ability to identify bug-inducing commits in the function history, in the file history and not in the file history.

Index Terms—SZZ, defect prediction, empirical study, ChatGPT.

Manuscript received 5 August 2023; revised 6 May 2024; accepted 9 May 2024. Date of publication 29 May 2024; date of current version 19 September 2024. This work was supported by the National Research Foundation, under its Investigatorship Grant NRF-NRFI08-2022-0002. Recommended for acceptance by J. Wang. (Corresponding author: Yunbo Lyu.)

Yunbo Lyu, Ratnadira Widyasari, and David Lo are with the School of Computing and Information Systems, Singapore Management University, Singapore 178902 (e-mail: yunbolyu@smu.edu.sg; ratnadiraw.2020@phdcs.smu.edu.sg; davidlo@smu.edu.sg).

Hong Jin Kang is with the University of California, Los Angeles, CA 90095 USA (e-mail: hjkang@g.ucla.edu)

Julia Lawall is with Inria-Paris, 75013 Paris, France (e-mail: julia.lawall@inria.fr).

Digital Object Identifier 10.1109/TSE.2024.3406718

I. INTRODUCTION

IN MSR 2005, Śliwerski, Zimmermann, and Zeller introduced an approach to identifying bug-inducing commits in a software code base [1]. Their approach, subsequently referred to as the *SZZ algorithm* [2], first identifies bug-fixing commits by correlating commits to reports in a bug tracker. It then uses the history mechanism of the version control-system to map each line of code removed by the bug fix to the commit that most recently added or modified that line. Finally, the resulting set of commits is *filtered* to remove those that postdate the bug report, unless there is evidence that the commit represents a partial fix or the commit is a bug-introducing commit for some other bug-fixing commit. The SZZ algorithm has been used in defect prediction techniques [3], [4], [5], [6], [7], and to analyze how bugs are born [8], [9], [10], [11].

Despite the success and influence of the SZZ algorithm, over the years, researchers have identified a number of weaknesses in its strategy for identifying bug-inducing commits. For example, the SZZ algorithm may incorrectly flag non-semantic source code (e.g., formatting changes) [2], meta-changes (e.g., branch changes) [12], and refactoring changes [13], [14] as bug-inducing. Several algorithm refinements have been proposed, such as excluding formatting and cosmetic changes [2], excluding changes that represent version-control system metadata (e.g., merge commits) [12], excluding changes that perform refactorings [13], [14], excluding non-executable changes [15], and combining the syntactic approach proposed by SZZ with analysis of changes in dependencies, including potentially interprocedural analysis [16].

Recently, attention has turned to the notion of a *ghost commit* [17]. As the SZZ algorithm relies on connecting the removed lines in the bug-fixing commit to the added lines in the bug-inducing commit, it is intrinsically unable to provide results for bug-fixing commits that have no removed lines (referred to as *Remove Mapping Ghost*), and to detect bug-inducing commits that have no added lines (referred to as *Add Mapping Ghost*). The SZZ algorithm furthermore cannot detect bug-inducing commits that are discarded due to the various filtering strategies (referred to as *Filtering Ghost*), such as commits dated after the bug report, commits that perform refactoring, or commits that address multiple purposes (i.e., tangled commits [18]). Some further variants have been proposed to mitigate these issues. Specifically, to address failures from Remove Mapping Ghost, A-SZZ [19] and Rezk et al.’s [12] method were proposed. To

address failures from Filtering Ghosts, Da Costa et al. [12] propose a framework to filter out suspicious commits.

Motivation: While there has been extensive research over two decades aiming to improve the precision of the SZZ algorithm [2], [12], [13], [14], [15], [16], discussions on enhancing recall are relatively limited. This analysis gap is critical because overlooking certain buggy patterns can impact downstream tasks dependent on this algorithm, such as defect prediction [5], [6], [11], [20], [21], [22], [23], [24] and the analysis of factors related to software quality [9], [10], [11], [25], [26], [27]. The closest work is that of Sahal et al. [19] and Rezk et al. [17], who use a syntax-based method [19] and context-aware data flow analysis [17] to address Remove Mapping Ghosts. However, their work only partially addresses one possible scenario where SZZ fails to identify bug-inducing commits. Moreover, there is a lack of comprehensive examination of the full range of scenarios where the SZZ algorithm falls short, such as cases where files are not shared between the bug-inducing and bug-fixing commits. Therefore, our work aims to present a whole picture of the recall challenges faced by all current SZZ algorithms.

The wide use of the SZZ algorithm and the introduction of its many variants raises the need to evaluate the algorithms fairly. The original SZZ lacked an empirical evaluation [1], while variants of SZZ were manually assessed by the authors themselves, rather than by domain experts. As noted by da Costa et al. [12] and Rosa et al. [28], due to a lack of developer-labeled data, evaluations of SZZ variants mostly rely on researchers' manual analysis of small datasets [1], [2], [14], [15], [16]. As the researchers are not developers of the projects being analyzed, their judgement on bug-introducing commits may not always be correct. The cost of manual analysis is also overwhelming. To avoid the need for manual analysis, while at the same time benefiting from the expertise of project developers, Rosa et al. [28] proposed an automation-based methodology to build a high-quality dataset from commits where the log message indicates both that the commit fixes a bug and the identity of the bug-inducing commit. Still, the resulting dataset only takes a few commits from each of many projects, which might not provide a realistic view. For instance, in Rosa et al.'s dataset, there exist just three bug-fixing commits from FFmpeg despite FFmpeg having over a hundred thousand commits.

There does exist, however, a large, developer-created dataset linking bug-fixing commits to the corresponding bug-inducing commits, as is needed to evaluate SZZ algorithms. Since October 2013, Linux kernel developers have been annotating bug-fixing patches with the keyword "Fixes:" followed by the commit id of the bug-inducing commit and the subject-line of the corresponding log message. Starting with fewer than 200 such annotated commits in 2013, the practice has steadily grown over the years, until reaching a steady state of 11-13K such commits per year, since 2019. Kernel developers include *Fixes:* annotations in their submitted patches, and the annotations are subject to the kernel code-review process. This dataset introduces the possibility of evaluating the SZZ algorithms at a large scale, and accurately measuring the impact of ghost commits.

Main Findings: In this paper, we collect a dataset of commits from the Linux kernel, named the Linux Kernel Fixing

Commits (*KFC*) dataset. The *KFC* dataset has more than 76K "Fixes:" commits over the last nearly 10 years of Linux kernel development to evaluate the SZZ algorithm and five of its variants. We find that all of the considered algorithms have recall and precision between 0.40 and 0.60, with some algorithms having higher recall, while others have higher precision, leading to an F1 score in all cases of around 0.50. The average recall for all SZZ algorithms evaluated in our dataset is 13.8% lower than that observed by Rosa et al. on their dataset, and the differences between individual SZZ algorithms are less pronounced.

Using the *KFC* dataset, we can additionally assess previous work on the frequency of ghost commits. Rezk et al. [17] have done a large scale study of the prevalence of ghost commits, focusing on 14 Apache projects. They found a median rate of 7.64% of occurrences of Remove Mapping Ghost (bug-fixing commits that have no removed lines, and thus have no changed lines to map back to bug-inducing commits) over the considered projects. In the Linux Kernel dataset, we find a much higher percentage, 17.65%. We also find a higher percentage than Rezk et al. of occurrences of Add Mapping Ghost (commits over the whole Linux commit history that have no added lines), with a rate of 5.3% for the Linux kernel as compared to a median of 2.68% for the applications in Rezk et al.'s dataset. Indeed, Rezk et al. lack a ground truth for the set of bug-inducing commits, and thus compute the rate of all commits that contain no added code as compared to the complete set of commits. In the case of the Linux kernel, we can also assess the rate of Add Mapping Ghost commits among the bug-inducing commits labelled by the developer. We find that Add Mapping Ghost bug-inducing commits account for only 0.7% of all bug-inducing commits (585 out of 79649). Thus, the results for the Linux kernel show that the findings of Rezk et al. across the entire code history do not reflect the actual rate at which commits that have no added lines are bug-inducing commits.

We furthermore find that ghost commits are not the only reason for failures (lower recall) of the SZZ algorithms. Indeed, we find that the bug-inducing commit is often not the most recent previous commit on any of the lines changed by the bug-fixing commit. And that over 13% of bug-inducing commits share no files in common with the subsequent bug-fixing commits. The files modified in the cross-file bug-inducing commits are present in the version control history but are different from the files in the bug-fixing commits. An example of a cross-file bug-fixing commit is commit 79b591c, which does not share any file with its bug-inducing commit. The bug-inducing commit modified a constant shared between multiple files, requiring a necessary unit conversion to be applied in each file that uses the constant. The bug was introduced as the bug-inducing commit neglected to update a relevant file. The correct unit conversion in this file was later introduced in the bug-fixing commit. The SZZ algorithms are founded on the assumption that "a bug-inducing change adds lines that are later removed by a fix." In the context of the git version control system, this means that SZZ algorithms use the *git blame* command just once for each removed line to identify the most recent change. To increase the recall, admittedly at the expense of precision, we examine whether bug-inducing commits might not always be in the most

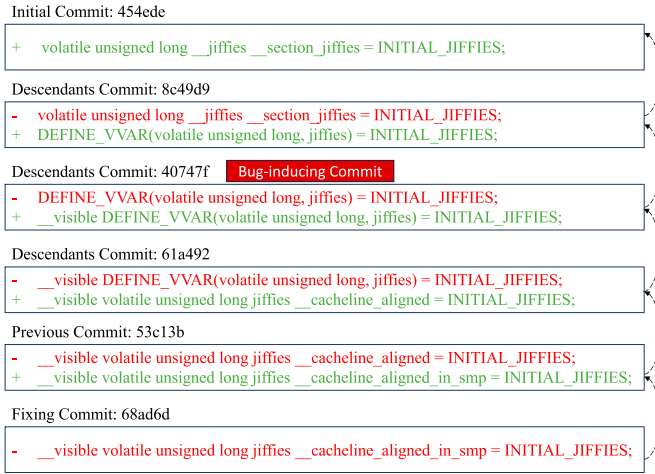


Fig. 1. The motivating example of Tracing-Commit SZZ.

recent change, but are rather concealed deeper in the change history.

To facilitate this exploration, we propose the Tracing-Commit SZZ (TC-SZZ) algorithm, which iteratively uses *git blame* until the initial commit is located, and then considers all commits in the change history as potential bug-inducing commits. Unlike all previous SZZ variants which identify only a single bug-inducing commit for a deleted line. This favors recall, but may increase the number of false positives as it returns commits in the line's change history that are not relevant to the bug. For example, as shown in Fig. 1, TC-SZZ returns all five commits in a line's change history as potentially bug-inducing commits, successfully identifying commit 40747f as the bug-inducing commit. In contrast, B-SZZ only considers the previous commit. Applying TC-SZZ to the 17,552 cases that are not detected by the SZZ algorithms but do not involve ghost commits revealed that 3,115 out of the 17,552 failure cases (17.7%) are located in the change history and not the most recent change. Algorithms that skip over refactoring changes have the potential to identify such bug introducing commits more accurately, but refactoring detection is language specific, and such approaches have only been developed for Java [29], [30], while the Linux kernel is entirely C code.

For the remaining 14,437 bug-fixing commits, 34.6% of the failure cases are found in the function history but not in the history of the lines that are changed by the bug-fixing commit. This means that the bug-inducing commit can be identified by applying *git log -L* on the changed functions (referred to as *Function-Log Detection*). For detailed examples, refer to Section VI-C. Additionally, 27.5% of failure are found in the file history but not in the history of the changed functions in that file. Here, the bug-inducing commit can be found by applying *git log* to the changed files (referred to as *Within-File Detection*). Finally, 37.9% of failure are in the history of a file other than the changed files (referred to as *Cross-File Detection*).

Experiments with ChatGPT: For the Function-Log Detection, Within-File Detection, and Cross-File Detection cases,

where the bug-inducing commit is not in the history of the lines changed by the bug-fixing commit, we experiment with ChatGPT. As ChatGPT has exhibited potential in addressing Software Engineering tasks [31], [32], we consider whether ChatGPT, specifically the GPT-4 version, can help SZZ algorithms identify bug-inducing commits for these failure cases. Specifically, we use the commit log *subject* and *body* of the bug-fixing commit, as well as all of the changed function's code, as input for ChatGPT. For Within-File Detection and Cross-File Detection, we leverage ChatGPT to help locate the bug-inducing function or file based on the bug-fixing commit, since the function or file responsible for the bug is not known from the bug-fixing commits. Our aim is for ChatGPT to pinpoint the specific line of code in the function that may have induced the bug and thus caused the bug-fixing commit. Given this line of code, we can then apply an iterated *git blame* to it to identify the bug-inducing commits. On 375 samples, for identifying bug-inducing commits Function-Blame Detection (a subcategory of Function-Log Detection, using *git blame* once for identifying the bug-inducing commit), ChatGPT showed increased efficacy in recall (with a precision of 0.28, recall of 0.62, and F1 of 0.39). However, when it came to dealing with failure cases both in and out of the file history, ChatGPT's performance was notably less successful. This is partly due to the challenge of a general-purpose Large Language Model in inferring function names or files not explicitly referenced in the bug-fixing commit.

Overview: In summary, our paper makes the following contributions:

- **Dataset Creation.** We identify the Linux kernel as a data source and collect a developer annotated and validated dataset of over 76K bug-fixing commits, which we have made publicly available.¹
- **Evaluation.** We conduct a comprehensive evaluation of six SZZ algorithms, revealing hidden dangers in real-world usage scenarios and analyzing the impact of ghost commits on the Linux kernel dataset.
- **Further Analysis.** We show that by utilizing TC-SZZ, we can address 17.7% of the failure cases. We classified the remaining failure situations as *Function-Log Detection*, *Within-File Detection*, and *Cross-File Detection*.
- **ChatGPT Efficacy Assessment.** We investigate the efficacy of ChatGPT in identifying bug-inducing commits in the three failure situations.

The rest of this paper is organized as follows. In Section II, we discuss the background and related work of our study. In Section III, we discuss how we build the dataset of bug-inducing commits. Section IV presents the design of our study. Section V introduces TC-SZZ. In Section VI, we show the evaluation results and answer our research questions. In Section VII, we explore the potential of ChatGPT to identify bug-inducing commits. Section VIII discusses the reason of SZZ failures. We discuss the threats to validity in Section IX. Finally, we draw the conclusions of our work and present future directions in Section X.

¹<https://doi.org/10.6084/m9.figshare.23889792.v2>

II. BACKGROUND AND RELATED WORK

In this section, we first provide an overview of the original SZZ algorithm, outlining its background and highlighting some limitations related to ghost commits. Next, we study the variants of the SZZ algorithm that have been proposed to improve its precision and recall. Lastly, we discuss the application and significance of the SZZ algorithm and its variants within the Software Engineering research community.

We focus on the eight SZZ algorithms that are considered by Rosa et al. [28]: B-SZZ [1], AG-SZZ [2], DJ-SZZ [15], L-SZZ [16], R-SZZ [16], MA-SZZ [12], RA-SZZ [13], RA-SZZ* [14], as well as some implementation variants: PyDriller (SZZ@PYD) [33], SZZ Unleashed [34], and OpenSZZ [35].

A. The Original SZZ Algorithm & Ghost Commits

SZZ, the name given to the algorithm initially introduced by śliwerski, Zimmermann, and Zeller [1], was developed to identify changes that induce bugs. In our forthcoming discussion, we will follow Rosa et al. [28] and refer to this original SZZ algorithm as B-SZZ. B-SZZ follows a core structure that is shared with its subsequent variants: (1) identifying bug-fixing commits, followed by a filtering process utilizing two confidence levels: *syntactic* and *semantic*; (2) mapping bug-fixing commits to their corresponding potential bug-inducing commits; and (3) eliminating potential bug-inducing commits that are unlikely to be the cause of bugs.

1) *Identifying Bug-fixing Commits*: In the initial stage, B-SZZ endeavors to establish a relationship between a bug-fixing commit in the version control system (VCS) and the corresponding bug reports in the issue-tracking system (ITS). To accomplish this, B-SZZ employs two independent confidence levels: a *syntactic* level, which infers links between a VCS log and a bug report, and a *semantic* level, which validates these links through bug report data.

For most software systems, there is no mandatory or default link between bug reports in the ITS and the VCS. Thus, developers may forget to add such links. Rezk et al. [17] define the missing bug-fixing commits that SZZ implementations fail to detect, where links were omitted, as **Ghost Commit 0 (GC 0)**. Herbold et al. [36] also find that half of the bug-fixing commits identified by SZZ are incorrect due to the mislabeled issues in the ITS (where developers incorrectly label issues as bugs, while some are merely improvements, etc.) [37]. Herbold et al. also find that SZZ misses about one-fifth of the bug-fixing commits.

This paper will not study the limitations of bug-fixing commit identification, as: (1) previous work [38], [39], [40] has already provided a comprehensive analysis and proposed tools to recover GC 0's missing links, and (2) the Linux kernel repository requires developers to use the "Fixes:" tag along with the first 12 characters of the SHA-1 ID when a patch addresses a bug in a specific commit. Therefore, we can collect a "developer-informed" Linux kernel dataset using the VCS without reliance on the ITS.

2) *Mapping*: Once the bug-fixing commits in the VCS have been identified, B-SZZ proceeds to the second step, where it

```

214 static int io_poll_check_events(struct io_kiocb *req, bool *locked)
215 {
    ...
249 +     if (io_is_uring_fops(req->file))
250 +         return IOU_POLL_DONE;
    ...
278 }
```

Fig. 2. A Remove Mapping Ghost (bug-fixing commit 7fdb5f).

```

3559 static int patch_nvhdmi_legacy(struct hda_codec *codec)
3560 {
    ...
3577 -     generic_acomp_init(codec, &nvhdm_i_audio_ops, nvhdmi_port2pin);
3578 -
3579 }
```

Fig. 3. An Add Mapping Ghost (bug-inducing commit 5a858e7).

maps the bug-fixing commits to potential bug-inducing commits, i.e., the commits that may have caused the bug. To do this, B-SZZ first employs CVS's *diff* command.² This command is analogous to *git show*, as it identifies the lines that have been deleted or modified by a commit. Then, B-SZZ invokes the *annotate* command (another CVS command, similar to *git blame*, but that does not identify the corresponding lines in earlier revisions) for each line to determine the most recent commit that deleted or modified the line prior to the fix. When there are multiple removed or modified lines, multiple commits can be mapped as potential bug-inducing commits. This paper focuses on issues in the mapping step.

Tools such as CVS *annotate* and *git blame* cannot map bug-fixing commits that do not modify or remove lines to bug-inducing commits. This situation is defined as **Mapping Ghost type 1 (MG 1)** by Rezk et al. (To give a more meaningful naming for MG 1, we name it **Remove Mapping Ghost** in this paper.) For example, in Fig. 2, bug-fixing commit 7fdb5f³ does not remove any lines and only added two lines in all the changes.

Similarly, if a bug-inducing commit does not add lines, it cannot be tracked by using *blame* on subsequently removed lines, as *blame* can only track previously existing lines. This situation is defined as **Mapping Ghost type 2 (MG 2)** by Rezk et al. (To give a more meaningful naming for MG 2, we name it **Add Mapping Ghost** in this paper.) For instance, in Fig. 3, bug-inducing commit 5a858e7⁴ does not add any lines and only removes two lines in all the changes.

3) *Filtering*: In the final stage, B-SZZ discards the potential bug-inducing commits that cannot have caused the bug, such as those committed to the VCS after the bug was reported. The potential bug-inducing commits that are not filtered out

²When B-SZZ was proposed, the Concurrent Versions System (CVS) was widely used for version control.

³<https://github.com/torvalds/linux/commit/7fdb5f014c3f71bc44673a2d6c5bb2d12d45f25>

⁴<https://github.com/torvalds/linux/commit/5a858e79c911330678b5a9be91a24830e94a0dc9>

during the filtering phase are reported as bug-inducing commits by B-SZZ.

Subsequent studies have proposed various filtering methods, including Content Filters [2], [41] and Suspiciousness Filters [12]. If all the potential bug-inducing commits are filtered out for a given bug-fixing commit, Rezk et al. refer to the bug-fixing commit as a **Filtering Ghost (FG)**.

We do not further examine the limitations of the filtering step in the SZZ algorithms for two reasons: (1) The filtering step heavily depends on data from the ITS, but we bypass any ITS by using the tags found directly in the Linux kernel commits; (2) The filtering process aims to improve precision by eliminating commits from the potential bug-fixing commits, while our focus is on enhancing the recall of the existing SZZ algorithm, i.e., adding potentially correct bug-inducing commits that were missing.

B. Improvements for B-SZZ

Despite the numerous proposals aimed at enhancing the accuracy of the SZZ algorithm [2], [12], [13], [14], [15], [16], the examination of bug-inducing commits not detected by SZZ is still in its infancy, and only limited efforts have been made to improve the recall [2], [17], [19].

Kim et al. [2] found that B-SZZ may incorrectly label bug-inducing commits due to considering formatting or cosmetic changes. To address this, they proposed AG-SZZ, which ignores such changes during the *mapping* process, improving the accuracy of locating bug-introducing commits. Moreover, as B-SZZ employs the CVS *annotate* command for mapping lines removed in bug-fixing commits to recent revisions, it cannot pinpoint the corresponding lines in earlier revisions. Kim et al. observed that such mappings might lead to inaccurately linked code segments. AG-SZZ, therefore, utilizes annotation graphs [42] to map line numbers across revisions, ensuring that both the starting and mapped lines belong to functions with the same name, improving the accuracy of the mapping process.

Williams et al. [15] propose DJ-SZZ to address the limitations of annotation graphs which cannot track individual lines through extensive modifications by using a line-number mapping approach [43]. Furthermore, they used DiffJ [44] to exclude non-executable changes.

Davies et al. [16] propose L-SZZ and R-SZZ, which improve on the AG-SZZ algorithm by selecting only one bug-inducing commit from all the candidate commits. L-SZZ selects the commit that changes the most lines. R-SZZ selects the most recent one.

Da Costa et al. [12] found that AG-SZZ identifies meta-changes (i.e., branch changes, merge changes, and property changes) as bug-inducing commits. AG-SZZ includes these commits as the annotation graph is unable to map the code between meta-changes and their preceding changes. To address this issue, Da Costa et al. propose MA-SZZ that connects all nodes in the annotation graph to their prior changes. This enhancement ensures that MA-SZZ does not identify meta-changes as bug-inducing commits.

Neto et al. [13] found that refactoring operations, which do not modify a program's behavior, impact the SZZ results. They extended MA-SZZ as two variants, RA-SZZ [13] and RA-SZZ* [14], to reduce the set of false positives reported by MA-SZZ. RA-SZZ and RA-SZZ* use RefDiff [29] and Refactoring Miner [30], respectively, to exclude refactoring operations from bug-inducing commit candidates, but both algorithms only work on Java files.

PyDriller (SZZ@PYD) [33], SZZ Unleashed [34], and OpenSZZ [35] are three open-source implementations of SZZ algorithms. PyDriller is a Python framework that analyzes Git repositories by extracting information about commits, diffs, and other information. It also implements B-SZZ and ignores C-style and Python-style comment lines. SZZ Unleashed uses line-number mapping without DiffJ, which allows it to work on non-Java files. OpenSZZ implements B-SZZ by using the *git blame* command.

To improve the recall of the SZZ algorithm, Sahal et al. [19] propose a syntax-based variation, A-SZZ, to address the case of bug-fixing commits that do not have removed lines (Remove Mapping Ghost). This algorithm traces changes for all lines within the code blocks that contain the additions. Rezk et al. [17] enhance the A-SZZ approach by incorporating context-aware data flow analysis, which includes mitigation strategies for specific programming scenarios such as null checks, creation of new entities, method overrides, logging operations, and class expansions. These strategies use data flow analysis on identifiers in added code lines, and then use B-SZZ to map the lines within these identifiers' data flow paths.

Table I presents the datasets that have been used for evaluating these algorithms, detailing the dataset type, the number of projects, and the bug-fixing commit count. In most cases, the ground truth is labeled by researchers without specialized domain knowledge. MA-SZZ uses an evaluation framework when there is no ground-truth label available between the bug-fixing commit and the bug-inducing commit. Specifically, it uses an automatic metric with three criteria to evaluate SZZ: earliest bug appearance, future impact of a change, and realism of bug introduction [12]. The only dataset labeled by developers contains, on average, only 1.2 commits per project, which may not sufficiently represent the entirety of issues arising in the development process of each project. This underlines the significance of collecting a dataset annotated by developers, encapsulating a range of problems that could emerge during a project's lifespan.

C. SZZ in SE Research

Many prior studies have discussed the significant role that SZZ plays in software engineering research [28], [45], [46].

SZZ is used as the first step in many studies, especially in the area of **defect prediction**. Many defect prediction studies use the SZZ algorithm to build datasets to train and evaluate their models [5], [6], [11], [20], [21], [22], [23], [24]. Herbold et al. [36] found that as of August 2019, 8 out of 15 of the public datasets used for defect prediction used B-SZZ to construct their ground truth. However, this approach suggests that the precision

TABLE I
DATASETS PREVIOUSLY USED FOR EVALUATING THE SZZ ALGORITHM. B-SZZ IS INDICATED AS //, AS THE AUTHOR DID NOT USE A DATASET FOR EVALUATION. “RE.” MEANS THE DATASET IS ANNOTATED BY RESEARCHERS. “DEV.” MEANS THE DATASET IS ANNOTATED BY DEVELOPERS

Approach / Dataset	#Dataset type	#Projects	#Bug Fixes
B-SZZ [1]	//	//	//
AG-SZZ [2]	Manually (re.)	2	301
DJ-SZZ [15]	Manually (re.)	1	25
L-SZZ & R-SZZ [16]	Manually (re.)	3	174
MA-SZZ [12]	Auto metrics	10	2,637
RA-SZZ [13]	Manually (re.)	10	365
RA-SZZ* [14]	Manually (re.)	10	365
Rosa’s Dataset [28]	Auto (dev.)	1,625	1,930

of the defect prediction model relies heavily on the quality of the dataset labeled by the SZZ algorithm [7]. A lot of mislabeled commits in the dataset will seriously affect the model prediction results, which highlights the importance of the accuracy of the SZZ algorithm.

The SZZ algorithm is also used to **analyze factors related to software quality** [9], [10], [11], [25], [26], [27]. A study by Eyolfson et al. [25] discusses the correlation between the time-based factors of a commit (such as the commit’s time of day, day of the week, and commit frequency) and the commit’s bugginess. Izquierdo-Cortázar et al. [27] and Tufano et al. [9] explore the correlation between the developer’s experience and the tendency to induce defects.

SZZ has also been used to identify bug-inducing commits at the pull request level in studies by Petruccio et al. [46] and Bludau et al. [47]. Furthermore, SZZ has been employed to determine the correct version information about vulnerable software within the National Vulnerability Database [48]. Given the importance and widespread use of SZZ, we aim to evaluate its performance in our study.

III. CONSTRUCTING A DATASET OF BUG-INDUCING COMMITS INFORMED BY DEVELOPERS

Evaluating the accuracy of the SZZ algorithm and its variants requires a dataset of bug-fixing commits accompanied in each case by the set of commits that introduced the fixed bug. śliwerski, Zimmermann, and Zeller [1] proposed experiments based on ECLIPSE and MOZILLA in which bug-fixing commits were chosen based on their references to a bug tracker; no evaluation was made of the accuracy of the corresponding collected bug-inducing commits. Subsequent SZZ variants were mostly evaluated using small, manually-validated datasets, as shown in Table I.

A. The Dataset of Rosa et al.

In 2021, Rosa et al. [28] proposed a new methodology to build a dataset for evaluating SZZ variants based on the observation that the log messages of some commits both contain a statement that the commit is fixing a bug *and* explicitly identify the commit that introduced the bug that is being fixed. For example, they have found the message “THRIFT-4513: fix bug

in comparator introduced by e58f75d” in the commit a8a97bd in the project apache/thrift.⁵ The presence of such messages raises the possibility of creating a large-scale dataset fully automatically, relying on the expertise of project developers to map bug-fixing commits to the corresponding set of bug-introducing commits. Nevertheless, they did not identify a standardized means of providing bug-fixing and bug-introducing commit information neither across projects nor within projects. Thus, they proposed to mine such information in commit logs using natural language processing (NLP) techniques, followed by manual verification. Using their approach, they created a dataset of 1,930 bug-fixing commits from 1,625 software projects written in a variety of programming languages. Using the resulting dataset, they compare the precision, recall, and F-measure of nine SZZ variants, and study the impact of dropping reported bug-inducing commits that postdate the associated entry in the bug tracker.

While the NLP-based approach of Rosa et al. [28] made it possible to obtain a large dataset, exploiting developer knowledge, researchers have identified some weaknesses in their results. The dataset remains small, containing 1,930 bug-fixing commits from 19,603,736 commits collected on GitHub, amounting to 0.001% of the total commits collected. While not all commits on GitHub are fixes or introduce bugs, previous studies [49], [50] have highlighted that 55% to 95% of the total cost of software development is devoted to maintenance. Therefore, it is likely that the 1,930 commits collected in the dataset is a vast underestimation of the actual number of bug fixes. And as there are on average only 1.2 commits per project, the collected commits may not be representative of the full scope of issues that arise in each project’s development.

B. A Dataset Extracted From the Development History of the Linux kernel

The Linux kernel was first released in 1991 and has grown to 23 MLOC as of the release of Linux v6.0 in October 2022. The Linux kernel has been developed using `git` since April 2005. Prior to October 2013, Linux kernel developers sporadically included in their commit log messages comments beginning with **Fixes:** followed by a reference to Bugzilla or some other URL. Starting with the commit 629c66a22c21⁶ from security researcher Kees Cook on October 24, 2013, the Linux kernel developer community has mostly settled on a strict format for bug-fix information. This format consists of one or more lines starting with the keyword **Fixes:**, each followed by the commit id of the bug-introducing commit, followed by the first line of the commit log message of the bug-introducing commit in parentheses. This format is encouraged by the Linux kernel documentation⁷ and by the Linux kernel “linting” tool `checkpatch`.⁸

⁵<https://github.com/apache/thrift/commit/a8a97bde9eada5ce0571ea1650b18f3ebc50e42>

⁶<https://github.com/torvalds/linux/commit/629c66a22c21b692b6e58b9c1d8fa56a60ccb52d>

⁷<https://www.kernel.org/doc/html/v4.9/kernel-documentation.html>

⁸<https://docs.kernel.org/dev-tools/checkpatch.html>

Using `git`, we collect 78,384 potential bug-fixing commits with **Fixes:**. We further mine all bug-inducing commits in each bug-fixing commit. In the first step, we use `[0-9a-f]{7,40}` to find the partial commit IDs (7-40 digits) of potential bug-inducing commits. We exclude partial commit IDs with fewer than seven characters to prevent ambiguity. In the second step, we search for the partial commit IDs in the list of actual complete commit IDs in the repository. If the partial commit ID can be found anywhere as a substring of a complete commit ID, we return the complete commit ID as the bug-inducing commit. To investigate the number of false positives in this step, we compare our results with those obtained using the `git rev-parse` command, which returns the full hash from a partial commit ID.⁹ As a result of this comparative analysis (of 104 cases), we observed two main issues that account for the differences between our method and the `git rev-parse` command: (1) **Overlapping Partial Commit IDs**: In certain instances, a partial commit ID is shared by multiple complete commit IDs, leading to false positives. We manually verified such cases and corrected 51. (2) **Typos in Commit IDs**: We found that the remaining 55 cases were the result of typos in commit IDs. Occasionally, commit IDs in the Linux kernel are incorrect due to typos, such as missing characters, which prevents `git rev-parse` from identifying the correct commit. For example, the partial commit ID `af4df655040` should actually be `3af4df655040`, making `git rev-parse` unable to find `3af4df65504088e9a7d20c0251e1016e521ad4fc`. Searching for partial commit IDs in the list accurately identified 55 of 56 cases. We encountered one instance where the commit did not belong to any branch in the repository, being part of a fork outside the repository. In the last step, we collect all the potential bug-fixing commits for which we can find at least one bug-inducing commit into the **Bug-fixing Commit Dataset**. We then collect those commits for which we cannot find any bug-inducing commits into the CSV file included in the replication package.

Our resulting Bug-fixing Commit Dataset contains 76,046 commits from between Oct 24, 2013 and Nov 20, 2022 that contain one or more properly formatted **Fixes:** annotations. The Linux kernel does not have an associated bug tracker through which we can validate that the annotated commits are fixing bugs. Nevertheless, the **Fixes:** annotation has, like the rest of the commit, been subject to the Linux kernel review process. Furthermore, we find that over 99% of the **Fixes:** annotations refer to existing commits.

We name our newly collected dataset **KFC** (Linux Kernel Bug-Fixing Commits). **KFC** contains a one-to-many relationship between bug-inducing commits and bug-fixing commits, as a single bug-inducing commit can lead to multiple future bug-fixing commits. Conversely, there is also a many-to-one relationship, where a single bug-fixing commit may be the result of multiple bug-inducing commits.

C. Verification of the Dataset

To verify the quality of our dataset, we conduct a manual analysis on a subset of the **KFC** dataset. This analysis aims to confirm that the “Fixes:” field is indeed used in practice to point to the cause of the bug, rather than mistakenly pointing to the initial implementation of related functionalities.

In this study, we followed four steps to validate the subset of the **KFC** dataset:

- (i) *Sampling*: The complete **KFC** dataset includes 76,046 bug-fixing commits. We selected a random sample of 383 cases to provide a statistically significant representation, aiming for a 95% confidence level with a 5% margin of error, according to standard sample size calculations.¹⁰
- (ii) *Preparation*: Three annotators initially discussed and refined their approach to ensure consistent labeling, based on an examination of a preliminary set of examples. This led to the creation of a comprehensive set of labeling guidelines.
- (iii) *Labeling*: Following these guidelines, the dataset was then labeled. One annotator was responsible for labeling all 383 cases, while the other two split the complete set, each labeling either 191 or 192 cases.
- (iv) *Discussion*: Upon completion of labeling, the annotators met to discuss and resolve all discrepancies.

After the *preparation* step, six categories were established for the labeling process: (1) “Yes”: The bug-inducing commit clearly points to the cause of the bug. (2) “Affected by other changes”: External modifications led to the emergence of the bug. (3) “Partial fix”: An intermediate commit attempted to fix the bug but did not completely succeed. (4) “Introducing a new function”: The commit introduced new functionality that was later involved in a bug, but was not directly the cause. (5) “No”: The bug-inducing commit does not point to the cause of the bug and does not fit into categories (2)-(4). Annotators are required to mark the reasoning behind this classification. (6) “Not sure”: The annotator could not determine whether the bug-inducing commit is related to the cause of the bug and is instructed to mark the reasons for this uncertainty.

After the *Labeling* and *Discussion* steps, we found that 81% (317 out of 383) of the sample cases fell into the “Yes” category. Additionally, classifications for the remaining cases were distributed as follows: 4% (15 out of 383) were categorized as “Introducing a new function,” 2% (9 out of 383) as “Affected by other changes,” and 3% (11 out of 383) as “Partial fix.” A further 2% (9 out of 383) of cases were labeled as “No,” as the annotators could not discern a relationship between the bug-fixing and bug-inducing commits. Lastly, 7% (26 out of 383) of the cases were tagged as “Not sure,” due to the annotators’ lack of expertise in the Linux kernel, which prevented them from making judgments. Despite a small percentage of cases not directly representing the cause of the bug, the vast majority of the dataset accurately reflects the intended use of the “Fixes:” field.

⁹<https://git-scm.com/docs/git-rev-parse>

¹⁰<https://www.surveymonkey.com/mp/sample-size-calculator>

IV. STUDY DESIGN

The purpose of this study is to empirically evaluate the various implementations of the SZZ algorithm on the currently largest dataset for SZZ evaluation and to investigate patterns that cannot be identified by SZZ (i.e., how to improve the recall of the SZZ algorithms), particularly Ghost Commits. To achieve these goals, we formulate three research questions:

RQ1: How well do the different variants of SZZ perform in identifying bug-inducing changes?

In our first research question, we aim to examine the performance of different SZZ variants on our dataset relative to a previous study by Rosa et al. [28]. Our study involves a dataset with more bug-fixing commits than in previous studies. Evaluating SZZ is crucial to ascertain if the prior studies' claims remain valid in an updated and more comprehensive dataset from a **complete** software project.

RQ2: What is the impact of Ghost Commits on the behavior of SZZ in a real software project?

In our second research question, we aim to explore the effects of ghost commits on the use of SZZ variants in a real software project. We divide RQ2 into two specific questions: (1) How frequently do ghost commits (Remove Mapping Ghost as bug-fixing commits, Add Mapping Ghost as bug-inducing commits) occur? (2) How do ghost commits impact the SZZ algorithms?

RQ3: What other situations, besides Ghost Commits, are undetectable by SZZ algorithms? In our final research question, we aim to identify additional situations where the SZZ algorithms fail, apart from the Ghost Commit situation. Given that SZZ algorithms are founded on the assumption that “a bug-inducing change adds lines that are later removed by a fix,” we address two specific questions within RQ3: (1) Do all failure cases, besides Ghost Commits, adhere to this assumption? (2) Beyond the previously studied situation, what other situations prevent SZZ from successfully identifying bug-inducing commits?

A. Algorithm Selection & Data Collection

We conduct experiments on several variants of the SZZ algorithm. Specifically, we adapt the main variants from the prior study by Rosa et al. [28] to run on our dataset. Of these, we do not use SZZ Unleashed since our commits are not associated with an issue tracker; thus there is no issue date. We do not use OpenSZZ since it needs a Jira URL as input. RA-SZZ* only works on Java as Refactoring Miner [30] can only detect refactoring operations in Java files. Therefore we do not use RA-SZZ* because the Linux kernel is a C-based project. In summary, we use B-SZZ, AG-SZZ, L-SZZ, R-SZZ, MA-SZZ, and PyDriller (SZZ@PYD). We used the **KFC** dataset, featuring the complete set of 76,046 **Fixes**: annotated bug-fixes.

B. Data Analysis

Given the set of bug-inducing commits predicted by the tested SZZ algorithms and the collected dataset, we evaluate the accuracy of the tested SZZ algorithms using recall and precision, two well-known Information Retrieval metrics [51].

We also use F1 score [52], which calculates the harmonic mean of recall and precision. These metrics have been used in previous studies to evaluate SZZ performance [28], [46]. The metrics for a bug-fixing commits are defined as follows, where $correct_{c_i}$ is the ground truth of bug-fixing commit c_i , i.e., the bug-inducing commits annotated by the original Linux kernel developer, $identified_{c_i}$ is the bug-inducing commits of bug-fixing commit c_i identified by the SZZ algorithms, and N is the total number of bug-fixing commits:

$$recall = \frac{1}{N} \sum_{c_i=1}^N \frac{|correct_{c_i} \cap identified_{c_i}|}{|correct_{c_i}|}$$

$$precision = \frac{1}{N} \sum_{c_i=1}^N \frac{|correct_{c_i} \cap identified_{c_i}|}{|identified_{c_i}|}$$

$$cF1 = 2 \cdot \frac{recall \times precision}{recall + precision}$$

We also analyze the complementarity of the SZZ variants following previous work [28]. Given the set of evaluated SZZ variants $SZZ_{eval} = \{v_1, v_2, \dots, v_n\}$, we compute their complementarity using the following metrics for each v_i [53]:

$$correct_{v_i \cap v_j} = \frac{|correct_{v_i} \cap correct_{v_j}|}{|correct_{v_i} \cup correct_{v_j}|}$$

$$correct_{v_i \setminus (SZZ_{exp} \setminus v_i)} = \frac{|correct_{v_i} \setminus correct_{(SZZ_{exp} \setminus v_i)}|}{|correct_{v_i} \cup correct_{(SZZ_{exp} \setminus v_i)}|}$$

$correct_{v_i}$ denotes the set of correct bug-inducing commits predicted by the SZZ algorithm v_i . $correct_{(SZZ_{exp} \setminus v_i)}$ denotes the correct bug-inducing commits predicted by all evaluated SZZ algorithms except for v_i . $correct_{v_i \cap v_j}$ represents the overlap between the correct bug-inducing commits identified by the SZZ algorithm v_i and another SZZ algorithm v_j . $correct_{v_i \setminus (SZZ_{exp} \setminus v_i)}$ denotes the correct bug-inducing commits that are only identified by the SZZ algorithm v_i .

To analyze the frequency of each Remove Mapping Ghost and Add Mapping Ghost, we use the same computation as Rezk et al. [17]. Remove Mapping Ghost: the percentage of bug-fixing commits that do not remove lines and only add lines out of all bug-fixing commits. Add Mapping Ghost: the percentage of commits that do not add lines and only remove lines out of all commits. With respect to Add Mapping Ghost, we also report the percentage of bug-inducing commits that do not add lines and only remove lines out of the set of bug-inducing commits, as these are the only commits that only remove lines that actually have an impact on the SZZ results.

V. TC-SZZ

The fundamental premise of the SZZ algorithm is that “a bug-inducing change adds lines that are later removed by a fix.” In other words, for a bug-fixing commit, the SZZ algorithms use the *git show* command to identify deleted or modified lines, and the *git blame* command to identify the most recent change to each such line. However, we discovered that some bug-inducing

commits are not the most recent changes before the bug fix, but rather appeared earlier in a deleted or modified line's change history.

For example, Fig. 1 shows a line's change history (i.e., iteratively using *git blame* until the initial commit) of the bug-fixing commit 68ad6d.¹¹ Following Bao et al. [48], we call the immediately preceding the lines changed in the fixing commit the **previous commits** (i.e., commit 53c13b in Fig. 1) and all commits (except the initial one) that previously modified these lines as the **descendant commits** (i.e., commit 8c49d9, 40747f, 61a492 in Fig. 1). We refer to the commit at which tracing can no longer continue as the **initial commit** (i.e., commit 454ede in Fig. 1). B-SZZ identified the *previous commit* as the bug-inducing commit, relying on using *git blame* once. Other algorithms, such as AG-SZZ and MA-SZZ, which improve on B-SZZ, access the rest of the descendant commits, identifying bug-inducing commits from other commits by considering criteria such as whether the commit is a meta-change. However, no existing algorithm comprehensively examines the line change history to identify the bug-inducing commit.

Therefore, we propose the Tracing-Commit-SZZ (TC-SZZ) algorithm, which assumes a distinct function as compared to previous SZZ algorithms. Unlike the six SZZ algorithms discussed in this paper, which normally identify only the *previous commit* as the bug-inducing commit, and V-SZZ [48], which considers the *initial commit* based on the observation that vulnerabilities are often foundational, TC-SZZ is designed to more effectively locate and understand the position of the bug-inducing commit in the change history of lines. TC-SZZ applies *git blame* iteratively to all deleted or modified lines in the bug-fixing commit until it reaches the *initial commit*, and returns all commits (i.e., the descendant commits and the initial commit in Fig. 1) in the change history of these lines. If *git blame* leads to a commit that has multiple removed lines, TC-SZZ chooses one among them to continue the iteration. Choosing all lines could lead to too much fanout, giving many false positives. For this, we have developed an algorithm based on V-SZZ [48]. Our algorithm takes into account both the string similarity and the relative distance between the line reported by *git blame* and the deleted lines in the same commit.

An alternative to the iterative use of *git blame* would be to use *git log -L*, which tracks changes to specific lines through the history. TC-SZZ, however, adds an additional dimension by considering string similarity, which can account for moved lines, thus identifying more bug-inducing commits than *git log -L*. Take for example commit 5b47348.¹² If we apply the *git log -L* command on the lines deleted by this bug-fixing commit, we only trace back to commit ed928a3.¹³ In this commit, the developer relocated a few functions to a different part of the file.

¹¹<https://github.com/torvalds/linux/commit/d8ad6d39c35d2b44b3d48b787df7f3359381dcbf>

¹²<https://github.com/torvalds/linux/commit/5b47348fc0b18a78c96f8474cc90b7525ad1bbfe>

¹³<https://github.com/torvalds/linux/commit/ed928a3402d8a24a70c242c63109c069a7b1f3ab>

TABLE II
RECALL, PRECISION, AND F1 RESULTS OF ALL SZZ ALGORITHMS

Algorithm	dataset _{linux}			dataset _{rosa}		
	Precision	Recall	F1	Precision	Recall	F1
B-SZZ	0.42	0.58	0.49	0.39	0.69	0.50
AG-SZZ	0.42	0.56	0.48	0.45	0.60	0.52
L-SZZ	0.57	0.43	0.49	0.52	0.45	0.48
R-SZZ	0.59	0.45	0.51	0.66	0.57	0.61
MA-SZZ	0.40	0.55	0.46	0.36	0.64	0.46
SZZ@PYD	0.43	0.55	0.48	0.39	0.67	0.49

The *git log -L* command interprets this as the initial commit for these lines. However, TC-SZZ goes beyond this by finding lines with similar content and executing *git blame* once more, enabling us to identify the bug-inducing commits.

To facilitate future research on understanding the placement of bug-inducing commits in the change history, TC-SZZ offers multiple modes: (1) **Chronological Trace Mode**: For every deleted or modified line in a bug-fixing commit, TC-SZZ returns all potential bug-inducing commits in the order of *git blame*. For instance, Fig. 1 illustrates the change history of a bug-fixing commit for a specific deleted or modified line, with TC-SZZ providing the sequence: previous commit, descendants commit 61a492, 40747f, 8c49d9, and initial commit. (2) **Unique Commits Mode**: For the sake of conciseness, based on the results from Chronological Trace Mode, only unique potential bug-inducing commits are returned, eliminating duplicates. (3) **Custom Blame Invocation Mode**: The user has the ability to specify the number of times *git blame* is invoked (a parameter of 1 is standard for SZZ, a parameter of 2 means using *git blame* twice for each modified line, and a parameter of -1 identifies the initial commit). Our implementation of TC-SZZ is publicly available in the replication package.

VI. RESULTS

In this section, we answer the three Research Questions proposed in Section IV.

A. RQ1: SZZ Performance

Table II presents the results for the six SZZ algorithms, ordered chronologically, on our newly collected **KFC** dataset, compared against the results reported by Rosa et al. [28] for their dataset (dataset_{rosa}).

The results demonstrate that R-SZZ is the best algorithm (with the highest F1) on both datasets, given that F1 combines precision and recall. There are several factors that cause R-SZZ to outperform the other SZZ variants on our dataset. First and foremost, R-SZZ only returns the most recent commit among all candidates. At the same time, 96.1% (73,109) of the Linux bug-fixing commits have only one bug-inducing commit. Thus, R-SZZ, by returning only one bug-inducing commit, achieves higher precision than SZZ algorithms that return multiple bug-inducing commits.

Four of the six algorithms demonstrate better precision on the **KFC** dataset than on the baseline (i.e., Rosa's result), with an

TABLE III
AVERAGE NUMBER OF BUG-INDUCING
COMMITTS IDENTIFIED BY VARIOUS SZZ
ALGORITHMS PER BUG-FIXING COMMIT

Algorithm	dataset _{linux}	dataset _{rosa}
B-SZZ	1.76	1.96
AG-SZZ	1.73	1.73
L-SZZ	1	1
R-SZZ	1	1
MA-SZZ	1.81	2.01
SZZ@PYD	1.73	1.95

average improvement of 9.67% for these four algorithms. Only AG-SZZ and R-SZZ have lower precision on the *KFC* dataset than on the baseline. R-SZZ and L-SZZ hold a significant advantage in precision compared to other SZZ variants on the *KFC* dataset. AG-SZZ, the third-best performer, performs in line with B-SZZ on the Linux dataset (42%). SZZ@PYD ranks as the fourth best algorithm in Rosa's dataset, but it is the third best algorithm in our dataset. Concurrently, MA-SZZ consistently underperforms.

Table III presents the average number of bug-inducing commits identified by the different SZZ algorithms for each bug-fixing commit across the two datasets, excluding the Ghost Commits. For instance, for each bug-fixing commit on Rosa's dataset [28], B-SZZ identifies an average of 1.96 bug-inducing commits, whereas, on the *KFC* dataset, B-SZZ only identifies an average of 1.76 bug-inducing commits. Thus, the precision of B-SZZ of 0.39 on Rosa's dataset increases to 0.42 on the Linux kernel dataset, an increase of 7.7%. The results in Tables II and III clearly show that as the number of identified commits decreases, precision increases.

Recall: As shown in Table II, all SZZ algorithms exhibit varying degrees of decline in recall as compared to the results on Rosa's Dataset, with an average decrease of 13.34% across the six algorithms. This highlights the challenge of achieving good recall with the SZZ algorithm, an issue that seems more significant on the Linux kernel dataset than in the previous empirical study [28]. Among all SZZ algorithms, B-SZZ consistently delivers the best performance on both datasets. In contrast, L-SZZ and R-SZZ are the least effective algorithms, as they identify only one potential bug-inducing commit each. MA-SZZ consistently ranks as the third-best algorithm. Notably, MA-SZZ, AG-SZZ, and SZZ@PYD demonstrate remarkably similar performance on the Linux dataset, with a mere 0.1% difference between them. One source of missed bug-inducing commits is ghost commits. In Section VI-B, we will examine the influence of ghost commits on the recall achieved by the SZZ algorithms. Then, in Section VI-C, we will investigate the other situations SZZ fails to identify bug-inducing commits.

Our findings confirm that the top-performing algorithms for precision and recall remain R-SZZ (~59%) and B-SZZ (~58%), respectively, which validates the previous research by Rosa et al. However, when applied to a complete repository such as the Linux kernel, the precision of R-SZZ and recall

TABLE IV
BUG-INDUCING COMMITTS CORRECTLY IDENTIFIED EXCLUSIVELY BY
EACH SZZ ALGORITHM

Algorithm	dataset _{linux}	percentage	dataset _{rosa}	percentage
B-SZZ	1024/47352	2.163%	21/784	2.679%
AG-SZZ	2/47352	0.004%	0/784	0%
L-SZZ	0/47352	0%	0/784	0%
R-SZZ	0/47352	0%	0/784	0%
MA-SZZ	160/47352	0.338%	1/784	0.128%
SZZ@PYD	66/47352	0.139%	0/784	0%

of B-SZZ, two leading SZZ algorithms, decline by 10.6% and 15.9%, respectively. While R-SZZ can still be useful for cases where precision is crucial and B-SZZ when recall is crucial, it is important to note that both algorithms still have limitations in accurately and comprehensively identifying bug-inducing commits in software repositories.

Table IV presents the $correct_{v_i \setminus (SZZ_{exp} \setminus v_i)}$ metric calculated for each SZZ variant v_i . This metric represents the correct bug-inducing commits identified exclusively by v_i and not by the other SZZ algorithms. For instance, consider the *KFC* dataset, where all SZZ algorithms combined correctly identify a total of 47,352 bug-inducing commits. Of these, B-SZZ uniquely and correctly identifies 1,024 bug-inducing commits not identified by other SZZ algorithms. Similarly, in the dataset_{rosa}, the total number of correctly identified bug-inducing commits by all SZZ algorithms is 784, with B-SZZ alone correctly identifying 21 bug-inducing commits that the other algorithms did not. In dataset_{linux}, four techniques (i.e., B-SZZ, AG-SZZ, MA-SZZ, SZZ@PYD) identify bug-inducing commits that remain undiscovered by other techniques, while in dataset_{rosa}, only two techniques (B-SZZ, MA-SZZ) exhibit this ability. Given that B-SZZ does not filter any potential bug-inducing commits and shows the highest recall in the results displayed in Table II, it has the most uniquely identified bug-inducing commits across both datasets. Interestingly, despite having the third-lowest recall, MA-SZZ can uniquely identify 160 bug-inducing commits.

The results depicted in Table IV illustrate the correct identification of bug-inducing commits by technique v_i that were missed by all other techniques. Conversely, Fig. 4(a) and 4(b) depict the $correct_{v_i \cap v_j}$ metric, which represents the degree of similarity between any two algorithms in their ability to accurately identify bug-inducing commits, calculated for each pair of SZZ variants on dataset_{linux} and dataset_{rosa} respectively. The heatmaps exhibit symmetry, as $correct_{v_i \cap v_j} = correct_{v_j \cap v_i}$. Fig. 4 reveals a significant overlap in the true positives identified in both datasets. In dataset_{rosa}, 13 out of 15 comparisons share more than 70% overlap in the identified true positives, while in dataset_{linux}, all 15 comparisons show this level of overlap. For instance, in Fig 4(a), the similarity between B-SZZ and AG-SZZ in the *KFC* dataset can be observed at the intersection of row 2 and column 1, or equivalently at row 1 and column 2 due to symmetry, where it is 0.94. The complementarity between the different SZZ variants is thus relatively low on both datasets, suggesting that there is a group of bug-fixing commits for which all variants fail to identify the correct bug-inducing commit.

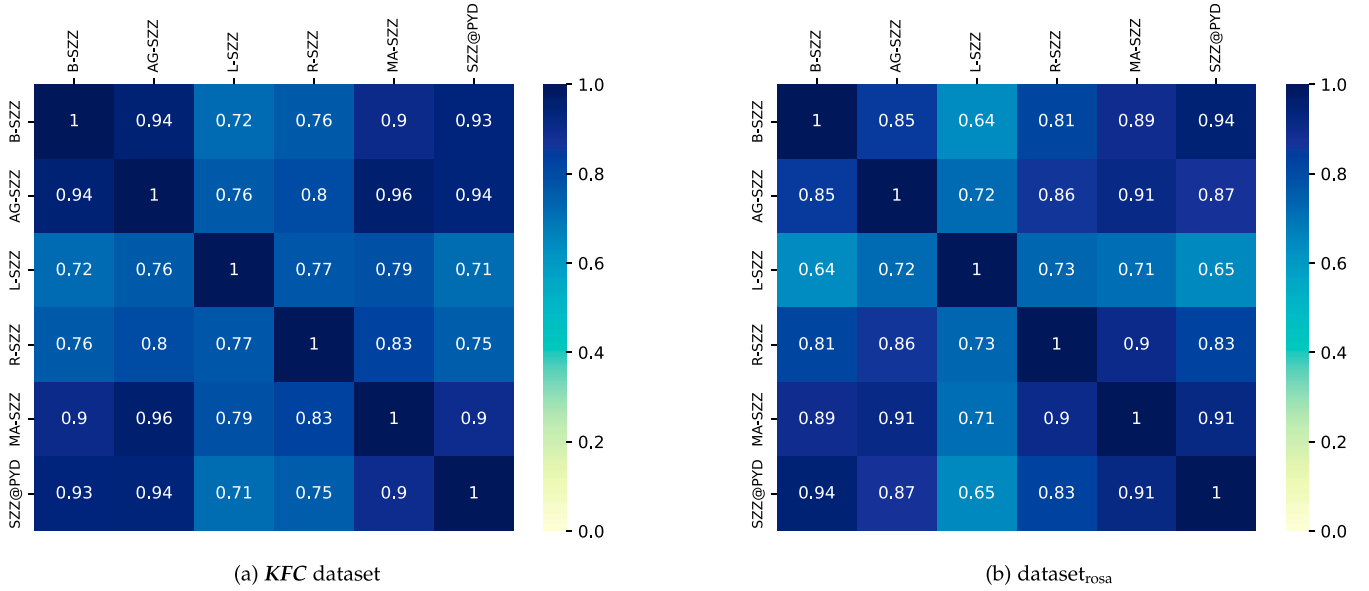
Fig. 4. Overlap between SZZ variants on *KFC* dataset and dataset_{Rosa}.

TABLE V
AN OVERVIEW OF THE PROJECTS AND GHOST COMMIT
FREQUENCY. RMG, AMG STAND FOR REMOVE MAPPING
GHOST AND ADD MAPPING GHOST

Project	Commits	RMG	AMG
Previous Mean [17]	12,216	8.05%	2.41%
Previous Median [17]	9,231	7.64%	2.68%
Linux kernel (Our study)	1,138,001	17.47%	5.30%

RQ1 Findings: Compared to Rosa’s dataset, the average precision in our dataset slightly improved from 0.46 to 0.47 ($\uparrow 2.2\%$), while the average recall decreased from 0.60 to 0.52 ($\downarrow 13.8\%$). Similar with the findings from the previous study [28], R-SZZ and B-SZZ remain the top performers in terms of precision and recall. The similarities between all the SZZ algorithms increase in the current dataset.

B. RQ2: Ghost Commits’ Impact

In this RQ, we aim to investigate the frequency of ghost commits in our dataset. Table V shows the ghost commit frequency from the dataset of Rezk et al. [17], derived from 12 Apache projects, as well as our study. The results reveal that in the Linux kernel, 17.47% of all bug-fixing commits are Remove Mapping Ghost (i.e., bug-fixing commits involving do not have removed lines). In the previous study by Rezk et al. [17], the highest percentage of Remove Mapping Ghost observed was 11.72%, with a median of 7.64% across the 12 Apache projects. Our Remove Mapping Ghost result is thus 128.66% higher than the median from the previous study, suggesting that Remove Mapping Ghost occurrences are more common in Linux kernel,

approaching one in six cases. This highlights the need to address this issue.

Add Mapping Ghost is also more prevalent in the Linux kernel compared to the datasets examined in prior studies. Rezk et al. reported the highest proportion of Add Mapping Ghost at 4.6%, with a median of 2.68%. However, in our *KFC* dataset, Add Mapping Ghost constituted 5.3% of all commits, which is 97.76% higher than the median observed in previous research. In Rezk et al.’s study, the ground truth of bug-inducing commits was unknown. Consequently, it was impossible to determine the number of bug-fixing commits for which SZZ algorithms could not identify their corresponding bug-inducing commits due to the Add Mapping Ghost problem. In contrast, our study has access to the ground truth, allowing us to more accurately assess the impact of Add Mapping Ghost. In our *KFC* dataset, we identified 482 bug-inducing commits as Add Mapping Ghost, which appeared 585 times in 584 bug-fixing commits. As a result, a mere 0.77% of bug-fixing commits are associated with an Add Mapping Ghost bug-inducing commit. In addition, Add Mapping Ghost bug-inducing commits account for only 0.73% of all bug-inducing commits (585 out of 79649). However, some bug-fixing commits associated with Add Mapping Ghost bug-inducing commits experience partial success (at least one bug-inducing commit is found) or failure due to Remove Mapping Ghosts. In the end, for 361 bug-fixing commits (0.47%), SZZ algorithms were unable to detect any bug-inducing commits due to Add Mapping Ghosts.

We also discovered Ghost Commits within Rosa’s dataset [28]. Given that previous work has not studied the Ghost Commit problem in a developer-informed dataset, we decided to investigate the Mapping Ghosts in Rosa’s dataset. We found that out of the 1115 bug-fixes, only 1084 bug-fixing commits’ repositories are still accessible on GitHub. Among these 1084 bug-fixing commits, 96 (8.86%) are Remove Mapping Ghosts;

TABLE VI
RESULTS AFTER REMOVING THE GHOST COMMIT. PRECISION, RECALL,
AND F-MEASURE CALCULATED FOR ALL SZZ ALGORITHMS. RMG,
AMG ARE THE SHORT FOR REMOVE MAPPING GHOST AND
ADD MAPPING GHOST

Algorithm	Filter RMG	Filter AMG	Filter RMG, AMG
B-SZZ	0.70	0.58	0.71
AG-SZZ	0.68	0.56	0.68
L-SZZ	0.52	0.44	0.53
R-SZZ	0.55	0.46	0.55
MA-SZZ	0.66	0.55	0.67
SZZ@PYD	0.67	0.56	0.67

from the 1946 bug-inducing commits, 184 (9.46%) are Add Mapping Ghosts.

To investigate the impact of ghost commits on SZZ algorithms, we remove the Remove and Add Mapping Ghost cases from *KFC* dataset. The experimental results for this modified dataset are presented in Table VI, which should be compared to the original results in Table II. “Filter RMG” refers to the removal of Remove Mapping Ghost bug-fixing commits from our dataset. “Filter AMG” indicates the removal of Add Mapping Ghost bug-inducing commits from our dataset. “Filter RMG, AMG” signifies the removal of both types of commits from our dataset. For example, when considering the B-SZZ algorithm, as shown in Table VI, only filtering Add Mapping Ghost reduces the recall from 0.71 to 0.58 ($\downarrow 22.4\%$), compared to filtering both Remove and Add Mapping Ghost. While filtering only Remove Mapping Ghost leads to a decrease in recall to 0.7 ($\downarrow 1.4\%$). These findings suggest that Remove Mapping Ghost has a more pronounced negative impact on the recall of SZZ algorithms. When compared to the results in Table II, the average recall for all SZZ algorithms in the *KFC* dataset improves by 22.2% after the removal of Remove and Add Mapping Ghost. The precision of all SZZ algorithms remains unchanged due to the SZZ algorithms not producing any results for Remove Mapping Ghost cases and the relatively small proportion of Add Mapping Ghost cases.

RQ2 Findings: Compared to Rezk’s study, the *KFC* dataset shows a significant increase in Remove Mapping Ghosts (from 7.64% to 17.47%) and Add Mapping Ghosts (from 2.68% to 5.3%). Notably, Remove Mapping Ghosts negatively impact the recall of the SZZ algorithms ($\downarrow 22.4\%$).

C. RQ3: Other Failure Situations

Given that B-SZZ exhibits the highest recall and identifies the most unique bug-inducing commits among all the SZZ algorithms we evaluated in this paper, we further scrutinize all failure instances of B-SZZ to uncover additional failure scenarios.

Table VII categorizes the B-SZZ algorithm results. “Success” indicates that B-SZZ identifies all bug-inducing commits for a bug-fixing commit; “Partial Success” implies at least one bug-inducing commit is identified but at least one is missed;

TABLE VII
CATEGORIZATION OF B-SZZ ALGORITHM RESULTS IN THE
LINUX KERNEL

Category	Number	Percentage
Success	43,846	57.66%
Partial Success	1,002	1.32%
Mapping Ghost	13,646	17.94%
Remove Mapping Ghost	13,285	17.47%
Add Mapping Ghost	361	0.47%
Failure without MG	17,552	23.08%
Within-File Detection	3,965	5.21%
Function-Log Detection	4,994	6.57%
Line Change Detection	3,115	4.1%
Cross-File Detection	5,478	7.2%

“Mapping Ghost” signifies that B-SZZ can not identify the bug-inducing commits due to the Mapping Ghost Problem. We eliminate bug-fixing commits classified as Success, Partial Success, and Remove Mapping Ghost from the *KFC* dataset, then search for Add Mapping Ghost cases among the remaining bug-fixing commits. If all bug-inducing commits for a specific bug-fixing commit are Add Mapping Ghost, we consider that B-SZZ’s failure is due to Add Mapping Ghost.

Out of 584 bug-fixing commits where at least one of their bug-inducing commits is Add Mapping Ghost, 361 fail as a result of Add Mapping Ghost. Upon completing these filters, we obtain a dataset consisting of 17,552 bug-fixing commits where B-SZZ fails, which we refer to as “Failures without MG.”

With our algorithm, using TC-SZZ, we find that out of the 17,552 bug-fixing commits in “Failure without MG,” 3,115 (17.7%) of those commits contain 3,163 bug-inducing commits within the change history of some removed line. In contrast, *git log -L* only manages to track down 2,800. We refer to the failures that can be solved by TC-SZZ as **Line Change Detections**.

For these 3,115 bug-fixing commits, the average length of the change history is 3.76, with the longest change history having length 35. Additionally, 1,272 bug-inducing commits occupy the second-closest position in the change history (with a total length ≥ 2), signifying that these bug-inducing commits can be located by using *git blame* twice. Furthermore, 1,489 bug-inducing commits are located at the furthest point in the change history, indicating that the initial code is faulty. The remaining 354 commits are in intermediate positions, i.e. descendant commits in Fig. 1.

To determine if the differences in performance between TC-SZZ and the other variants of SZZ are significant, we performed a statistical analysis on the *KFC* dataset, which comprises bug-fixing commits in the Linux kernel from 2013 to 2022. We divide the dataset by year to reflect the yearly increase of new bug-fixing commits. Given the low number of bug-fixing commits in 2013 (152), we merged the data from 2013 and 2014 into a single subset for analysis. Initially, we used the Shapiro-Wilk test to confirm that the recall data for all algorithms follow a normal distribution. Subsequent t-tests were performed to compare the recall metric of TC-SZZ with other SZZ algorithms. As shown in Table VIII, there are significant differences

TABLE VIII
INDEPENDENT T-TEST RESULTS COMPARING TC-SZZ WITH
OTHER SZZ ALGORITHMS IN TERMS OF RECALL,
INCLUDING EFFECT SIZES

Comparison	T-Statistic	P-Value	Effect Size
TC-SZZ vs B-SZZ	10.3439	< 0.0001	4.876
TC-SZZ vs AG-SZZ	15.5688	< 0.0001	7.339
TC-SZZ vs L-SZZ	47.7760	< 0.0001	22.522
TC-SZZ vs MA-SZZ	18.6914	< 0.0001	8.811
TC-SZZ vs PD-SZZ	18.4269	< 0.0001	8.687
TC-SZZ vs R-SZZ	30.8874	< 0.0001	14.560

in recall between TC-SZZ and other SZZ algorithms, with all p-values significantly less than 0.05. Furthermore, we calculated the effect size using Cohen’s d [54], as reported in the fourth column of Table VIII. The effect sizes are notably large across all comparisons—with Cohen’s d values exceeding 2.0—signifying substantial differences in recall between TC-SZZ and the other SZZ algorithms. Our analysis reveals that there is a significant difference every year between the performance of TC-SZZ and the other SZZ variants.

Given that TC-SZZ can resolve 17.7% of failure cases, we check whether we can pinpoint even more bug-inducing commits by using a TC-SZZ-like approach. We apply *git log* to all files affected by the bug-fixing commit, and then apply *git log -L* to all code lines within the function containing the deleted/modified lines from the bug-fixing commit.

We further define three new types of failure cases:

- (1) **Function-Log Detection:** This refers to situations where the bug-inducing commit is located in the history of the function containing deleted code, as identified using the command *git log -L*. For instance, in the bug-fixing commit *cbcf099*,¹⁴ two lines were deleted and two lines were added. Despite this, neither the SZZ algorithm (B-SZZ) nor the use of *git blame* on all of the lines in the function that contains the deleted line can identify the bug-inducing commit. However, by applying *git log -L* to all of the lines in the function that contains the deleted lines, we are able to successfully identify the bug-inducing commit.
- (2) **Within-File Detection:** This term refers to the identification of bug-inducing commits within the history of a specific file, particularly the one that was modified in a bug-fixing commit where lines of code were deleted. For example, in commit *d53c51f*,¹⁵ methods like the SZZ algorithm and the use of *git log -L* on the function that contains the deleted lines can not identify the bug-inducing commit. Specifically, the line of code introduced by the bug-inducing commit is not the same as the one modified in the bug-fixing commit, although they are in the same file. In contrast, by applying *git log* to the entire file that encompasses these deleted lines, we can successfully uncover the bug-inducing commit.
- (3) **Cross-File Detection:** This term describes instances where the files are not shared between the bug-inducing

and bug-fixing commits. Specifically, the bug-inducing commit cannot be located within the same file as the deleted code when using the *git log -L* command. As a result, since the files between bug-inducing and bug-fixing commits are not shared (as required by SZZ), it is challenging to reach such bug-inducing commits given the bug-fixing commits. For example, in the bug-fixing commit *d15d731*,¹⁶ all modifications (both additions and deletions) occur in the file “drivers/base/firmware_loader/main.c”. However, the bug-inducing commit *f531f05*¹⁷ only modifies the file “drivers/base/firmware_class.c”. This discrepancy prevents the SZZ algorithm from identifying the bug-inducing commit. As a result, we introduce a new category of Mapping Ghosts known as “Cross-File Mapping Ghosts” to capture these scenarios where the bug-inducing changes are not found in the same file as the bug-fixing changes.

The results in Table VII reveal that out of the remaining 14,437 failure cases, 4,994 (34.6%) are Function-Log Detection, 3,965 (27.5%) are Within-File Detection, and 5,478 (37.9%) are Cross-File Detection. Note that the category Within-File Detection comprises only cases that are in the file history but are not in the function history. For the 37.9% of failure cases that are not present in the file history, unless the bug-fixing commit explicitly mention the bug-inducing commits or specify a particular file, it becomes challenging for a researcher without domain expertise to correctly identify the bug-inducing commits. This task becomes even more difficult for an automated approach.

RQ3 Findings: Our findings show that our proposed TC-SZZ can successfully address 17.7% of failure cases. 5,478 of 14,437 (37.9%) failure cases have bug-inducing commits that are not in the file history, making them hard to resolve.

VII. CLASSICAL MEETS MODERN: ENHANCING SZZ WITH CHATGPT

In this section, we attempt to address three types of failure situations (*i.e.*, Function-Log Detection, Within-File Detection, and Cross-File Detection) utilizing ChatGPT, based on the GPT-4 version, which is one of the most sophisticated Large Language Models (LLMs) currently available.

Given that ChatGPT is adept at addressing Software Engineering tasks [31], [32], we aim to investigate whether it can assist researchers or developers in more effectively identifying bug-inducing commits. We interacted with GPT-4 through web chat using the default settings. OpenAI does not release the specific settings it employs for the web chat. Both the prompt and the corresponding results from ChatGPT are available in the replication package. We conducted the experiment from June 28th to July 4th, 2023 and from November 30th, 2023 to Jan 3rd, 2024.

¹⁴<https://github.com/torvalds/linux/commit/cbcf099ae33e7a8e1dba7ca935556634f679ccf>

¹⁵<https://github.com/torvalds/linux/commit/d53c51f26145657aa7c55fa396f93677e613548d>

¹⁶<https://github.com/torvalds/linux/commit/d15d7311550983be97dca44ad68cbc2ca001297b>

¹⁷<https://github.com/torvalds/linux/commit/f531f05ae9437df5ba1ebd90017e4dd5526048e9>

Considering that GPT-4 currently has a cap of 25 messages every 3 hours and the manual process involved, we selected a sample of 375 failure cases¹⁸ (with a 95% confidence level and 5% margin of error, following previous work [55], [56], [57], [58]) from the total of 14,437 identified failure cases. We exclude the “Line Change Detection” category because TC-SZZ was able to identify all bug-inducing commits in this group. To simplify, we do not use TC-SZZ in this section, as our ground truth in Section VI-C only uses *git log -L*.

In Section VI-C, we classified the 14,437 failure cases into three categories: Function-Log Detection, Within-File Detection, and Cross-File Detection. This section introduces an additional category, **Function-Blame Detection**, which is a subset of Function-Log Detection. The differentiation lies in the method used for identifying the bug-inducing commit. In the case of Function-Log Detection, the *git log -L* command is employed. This command helps in tracing the evolution of specific parts of a file, revealing the historical changes that led to the current state. For **Function-Blame Detection**, we utilize the *git blame* command only once. This command allows us to pinpoint the last change made to those lines. There are a total of 3,200 Function-Blame Detection cases and 4,994 Function-Log Detection cases. For instance, consider the commit 06bbf75.¹⁹ This commit involves a single deleted line within the function “process_response_list”. By applying *git blame* to all the lines of this function, we can identify the commit that initially introduced the bug. Although *git log -L* can also be used for this purpose by showing the evolution of the function over time, *git blame* specifically returns only the most recent changes.

In our sample of 375 cases, we found that 8.5% (32 out of 375) are **Function-Log Detections** (excluding Function-Blame Detection cases), and 25.1% (94 out of 375) are **Function-Blame Detections**. Moreover, 30.9% (116 out of 375) of the cases are **Within-File Detections**, while 35.5% (133 out of 375) of the cases are the **Cross-File Detections**.

A. Pipeline of ChatGPT Experiment

Fig. 5 shows the pipeline of the ChatGPT experiment. The pipeline has three steps: (A) *Cross-File Detection*, (B) *Within-File Detection* (C) *Function-Log (Function-Blame) Detection*. The prompts used in the different steps are shown in Table IX.

For the **Function-Log Detection** cases, we use the Prompt C in the Table IX to query ChatGPT. Specifically, for the ⟨Commit Message⟩ in the table, we use the commit log *subject* and *body* of the bug-fixing commit. For ⟨Code of the Function⟩, we use the code of all changed functions as input for ChatGPT. Subsequently, we execute *git log -L* on the line(s) of code suggested by ChatGPT to trace the bug-inducing commits. Similarly, for the **Function-Blame Detection** cases, we apply the *git blame* command to the line(s) of code that ChatGPT indicates to

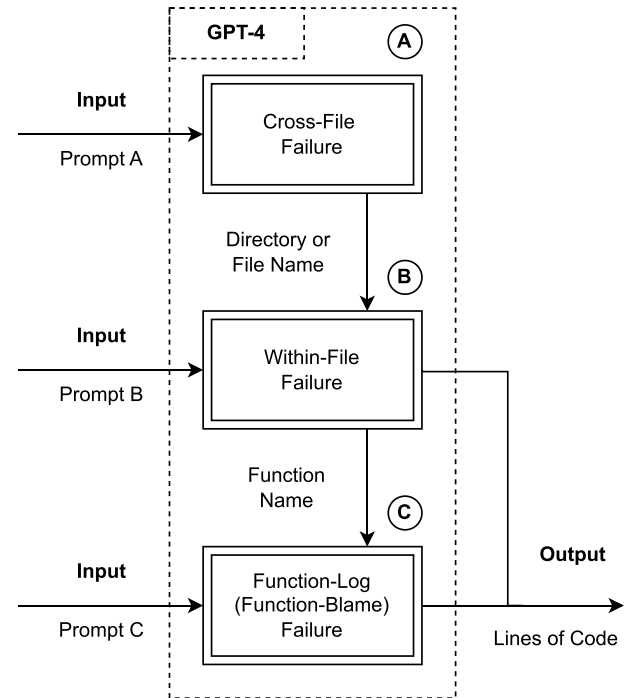


Fig. 5. The Pipeline of the ChatGPT Experiment.

TABLE IX
PROMPT FOR GPT-4 IN DIFFERENT SITUATIONS

Situations	Template
Prompt A: Cross-File Detection	⟨Commit Message⟩ Based on the above commit message of a bug-fixing commit, which file in the Linux kernel could be causing this bug-fixing commit?
Prompt B: Within-File Detection	⟨Commit Message⟩ Based on the above commit message of a bug-fixing commit and function names in file ⟨file_name⟩, which function or functions could be causing this bug-fixing commit? ⟨Function Name List⟩
Prompt C: Function-Log (Function-Blame) Detection	⟨Commit Message⟩ Based on the above commit message of a bug-fixing commit, please identify the line or lines of code in the following code that could be causing this bug-fixing commit: ⟨Code of the Function⟩

identify the bug-inducing commits. To ensure fairness in the experiment, we remove all portions of the commit message that directly reference a bug-inducing commit.

Figs. 6 and 7 show an example where ChatGPT successfully aids in identifying a bug-inducing commit from a Function-Blame Detection case.²⁰ Initially, we query ChatGPT using the prompt depicted in Fig. 6. Subsequently, the line(s) of code returned by ChatGPT (shown in Fig. 7) are subjected to the *git blame* command to get the bug-inducing commits.

¹⁸This sample size was determined to be statistically representative using a popular sample size calculator (<https://www.surveymonkey.com/mp/sample-size-calculator/>) with a confidence level of 95% and a 5% margin of error.

¹⁹<https://github.com/torvalds/linux/commit/06bbf753476dab23eb262cb5fba6d6d277a0ba3>

²⁰<https://github.com/torvalds/linux/commit/761e205b559be52852d85e0db4a034c9f57965f9>

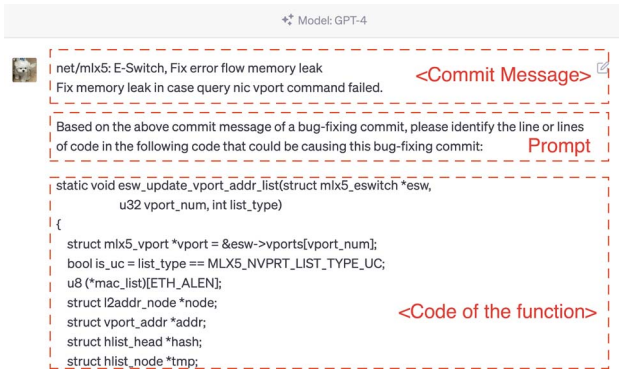


Fig. 6. Input Provided to ChatGPT for Commit 761e205 in Linux kernel. In practice, we provide the entire function code to ChatGPT, however, space constraints prevent us from including the full screenshot in the paper. The complete screenshot is available in the replication package.

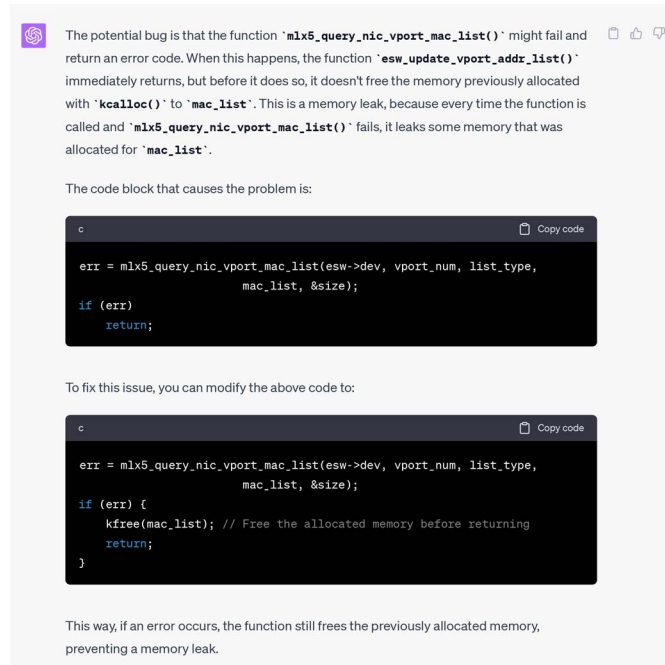


Fig. 7. ChatGPT's Output for Commit 761e205 in Linux kernel. If the details in the figure are difficult to discern, please refer to our replication package.

For the **Within-File Detection** cases, we follow a two-step method: (1) We first use *ctags*²¹ to extract all function names in a file. Subsequently, we pose the Prompt B in Table IX to ChatGPT. We provide ChatGPT with the function names extracted using *ctags*, instead of the entire code, because the number of code tokens in a file often exceeds the token limit set by ChatGPT. (2) We then apply the same method as in the Function-Log Detection category for any correctly identified function to find the bug-inducing commits.

For the **Cross-File Detection** cases, we follow the three steps shown in Fig. 5: (A) The initial step involves identifying the file containing the bug-inducing commits. We pose the Prompt

²¹<https://ctags.io>

A in Table IX to ChatGPT. Given that the Linux kernel has 15,000 files and limited information is available in the commit message, ChatGPT might have difficulty identifying an exact file. We have observed that it could instead return a directory, despite not having been prompted to do so, containing the code affected by the bug-inducing commit. Hence, we evaluate ChatGPT's performance based on the correctness of the identified directory and file. (B) If ChatGPT correctly identifies the file, we adopt the same method as the first step in the **Within-File Detection** category to determine the function. (3) If the correct function is determined by ChatGPT, we proceed with the same method as for the **Function-Log Detection** category to identify the bug-inducing commits.

B. Results

Since the **Function-Blame Detection** category shares similar principles with the SZZ algorithm, we adopt the SZZ algorithm's evaluation methodology, as detailed in Section IV-B, to compute the precision, recall, and F1 for this category. In contrast, the evaluation for the remaining three categories (i.e., **Function-Log Detection**, **Within-File Detection**, **Cross-File Detection**) diverges from the evaluation of the **Function-Blame Detection**. Instead of adhering strictly to SZZ-based principles, for these categories, we focused on assessing the number of bug-fixing commits that could find their bug-inducing commits (i.e., Recall) with the assistance of ChatGPT. We also used the *git log -L* command to identify as many bug-inducing commits as possible, aiming to evaluate the potential of a *git blame* based approach augmented with ChatGPT. For the Within-File Detection category, we evaluate three indicators (Function, Line of code, and Recall) as returned by ChatGPT. For the Cross-File Detection cases, we evaluate five indicators: Directory, File, Function, Line of code and Recall.

The result of the ChatGPT experiment is shown in Table X. For Function-Blame Detection, ChatGPT exhibits a high recall of 0.62, comparable to that of B-SZZ on the complete Linux dataset. The precision is, however, only 0.28, because ChatGPT selects from lines of all changed functions, resulting in a substantially larger selection space, especially given that some functions span hundreds of lines. For **Function-Log Detection**, the recall of our ChatGPT method dips to 0.31. Regarding **Within-File Detection**, ChatGPT successfully predicts the correct function in 32 out of 76 cases (42.1%). In 44 instances, ChatGPT makes an incorrect prediction, and in ten other cases, the prediction fails due to token limitations, no functions, or ChatGPT's inability to discern the correct function. **Cross-File Detection** presents a significant challenge as the commit messages often lack sufficient information to identify the file. In over half the cases, ChatGPT could only return the directory of the file that might contain the bug-inducing commits. However, we found that ChatGPT correctly predicts the directory in 65% of these cases. Furthermore, it successfully identifies bug-inducing commits in 11 cases where the bug-inducing commit is not in the file modified in the bug-fixing commit. In analyzing the 11 successful cases of Cross-File Detections, we observed that the majority of the pairs of bug-fixing and bug-inducing

TABLE X
RESULTS OF THE CHATGPT EXPERIMENT // INDICATES THAT THE RESPECTIVE INDICATOR WAS
NOT EVALUATED IN THE CORRESPONDING CATEGORY

Failure Situations	Number	Directory	File	Function	Line of Code	Recall	Precision	F1
Function-Blame Detection	94	//	//	//	//	0.62	0.28	0.39
Function-Log Detection	32	//	//	//	✓: 8 ✗: 24	0.31	//	//
Within-File Detection	116	//	//	✓: 32 ✗: 44	✓: 25 ✗: 26	0.22	//	//
Cross-File Detection	133	✓: 86 ✗: 46	✓: 43 ✗: 46	✓: 10 ✗: 11	✓: 11 ✗: 5	0.08	//	//

commits are located within the same directory. For example, the bug-fixing commit 5b8481f and the bug-inducing commit 89a23c8 are both found within the ‘net/ipv6’ directory. Nevertheless, there are four instances where the bug-fixing commit is not even in the same root directory as the bug-inducing commit.

Through this ChatGPT experiment, we conclude that ChatGPT exhibits potential in assisting researchers and developers in identifying bug-inducing commits. We acknowledge that this exploration is only preliminary, given that our experimental setup does not reflect realistic conditions, as we knew the categories beforehand. Nonetheless, considering that ChatGPT is a generalized model not specifically fine-tuned for this task, its performance hints at the potential benefits of similar LLMs. We hope that this work will catalyze further research efforts on using LLMs to overcome the limitations of the SZZ algorithm in identifying bug-inducing commits, including challenges such as ghost commits and cross-file detection.

C. Evaluating ChatGPT Beyond the 2021 Data Cutoff

ChatGPT may have been trained on the data used in our experiment, which could bias the results. Our initial work with ChatGPT, involving 165 commits, was carried out from June 28th to July 4th, 2023, prior to OpenAI’s announcement on September 27th, 2023 that it had enhanced ChatGPT’s capabilities, extending its knowledge base beyond the previous cutoff in September 2021.²² This raises the opportunity to compare the success rate ChatGPT on those commits that were released prior to September 2021 to those that were released after that date, that were certain to be outside of the knowledge base of ChatGPT.

In an analysis of 165 commits up to September 2023, 134 were made before September 2021, and the remaining 31 occurred after this date. This date serves as the data cutoff point, dividing the commits into two groups for evaluation: commits before the cutoff may have been part of the ChatGPT training data, while those subsequent to the cutoff had not. As shown in Table XI, for Function-Blame Detection, testing cases subsequent to the data cutoff exhibited a notable improvement with ChatGPT, achieving a 24% increase in recall (from 0.47 to 0.71) and a 7% increase in precision (from 0.26 to 0.33). However, for Function-Log Detection and Within-File Detection, testing cases subsequent to the data cutoff resulted in a 2% decrease in performance. Regarding Cross-File Detection, testing cases before the data cutoff had an 8% recall rate, whereas cases after the data cutoff failed to identify the correct bug-inducing commits in this category. Given that testing cases after the data

TABLE XI
COMPARATIVE OF CHATGPT ON BEFORE AND AFTER 2021 DATA CUTOFF

Failure Situations	Before (134)		After (31)	
	Recall	Pr.	Recall	Pr.
Function-Blame Detection	0.47	0.26	0.71	0.33
Function-Log Detection	0.35	-	0.33	-
Within-File Detection	0.24	-	0.22	-
Cross-File Detection	0.08	-	0	-

cutoff demonstrated a significant advantage of up to 24% in one category, and no substantial disadvantage in the three remaining categories—with drops ranging from 2% to 8%—there is no conclusive evidence to suggest that testing cases potentially seen by ChatGPT would impact the experimental results.

D. Compare With Code-Specific Models

To further evaluate the performance of ChatGPT, we conduct a comparative analysis with the code-specific models CodeT5+ [59] and CodeGen2.0 [60], applying a consistent experimental setting as outlined in Section VII-A. We follow again the evaluation methodology described at the beginning of Section VII-B. We use the same prompts for ChatGPT, CodeT5+, and CodeGen2.0 to ensure a fair comparison. CodeT5+, proposed by Wang et al. [59], stands out as an encoder-decoder code foundation LLMs, demonstrating good performance across more than 20 code-related benchmarks involving both understanding or generative tasks, including in the zero-shot setting. Given that the 2B, 6B, and 16B parameter sizes of CodeT5+ are built upon CodeGen, we also include a comparison with CodeGen2.0 in our analysis. However, due to constraints related to GPU memory, our assessment is confined to the codet5p-2b and codegen2-3_7B models for the inference stage, as these are the largest model sizes we can load. Both models are deployed on a Docker environment with NVIDIA RTX 6000 Ada Generation (48GB of graphics memory). We use the same settings for the CodeT5+ 2B²³ and CodeGen2-3.7B²⁴ models from Hugging Face. The default configurations for CodeT5+ include parameters such as temperature of 1.0, model type as encoder-decoder, and 16-bit floating point tensors. On the other hand, CodeGen2 uses similar defaults with a temperature of 1.0 but employs the GPT2Tokenizer and configures tensors to use 32-bit floating point. Additionally, for both models, we adjust the max_length parameter to 2048.

²³<https://huggingface.co/Salesforce/codet5p-2b>

²⁴https://huggingface.co/Salesforce/codegen2-3_7B_P

²²<https://twitter.com/OpenAI/status/1707077710047216095?s=20>

```

Model: GPT-4

net/sched: act_nat: validate the control action inside init()
the following script:

# tc qdisc add dev crash0 clsact
# tc filter add dev crash0 egress matchall \
> action nat ingress 1:18.2.2 pass index 90
# tc actions replace action nat \
> ingress 1:18.2.2 goto chain 42 index 90 cookie c1a0c1a0
# tc actions show action nat

... reaming commit message of bug-fixing commit

CR2: 0000000000000000

Validating the control action within tcf_nat_init() proved to fix the
above issue. A TDC selftest is added to verify the correct behavior.

Based on the above commit message of a bug-fixing commit, which file in the Linux kernel
could be causing this bug-fixing commit?

```

(a) Same input for GPT-4, CodeT5+, and CodeGen2.0.

```

Based on the information provided, the bug seems to be related to the network scheduler
('net/sched'), specifically in the NAT (Network Address Translation) action handling which
is used for packet scheduling and shaping in the Linux kernel.

Therefore, the file that is likely causing this issue is 'act_nat.c' located in the network
scheduling subsystem of the Linux kernel, i.e., in the directory 'net/sched/'.

The function 'tcf_nat_init()' is the initialization function for the NAT action in the
network scheduler, and since the commit message says "Validating the control action within
tcf_nat_init() proved to fix the above issue", it's highly probable that this function resides in
'act_nat.c'.

However, I recommend examining the complete codebase or using the specific commit hash
to confirm this, as the exact file can vary between different versions of the Linux kernel.

```

(b) Output of GPT-4 showing correct directory identification.

```

Based on the above commit message of a bug-fixing commit, which
file in the Linux kernel could be causing this bug-fixing commit?

#PF error: [normal kernel read fault]
#PF error: [normal kernel read fault]
#PF error: [normal kernel read fault]
#PF error: [normal kernel read fault]
#PF error: [normal kernel read fault]

```

(c) Output of CodeT5+ displaying an error message without providing the filename.

```

Based on the above commit message of a bug-fixing commit, which file in the Linux kernel
could be causing this bug-fixing commit?

/*
<|c|>/*
 * Copyright (c) 2017-2019 THL A29 Limited, a Tencent company. All Rights Reserved.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
#

```

(d) Output of CodeGen2.0 presenting unrelated license content.

Fig. 8. Comparative Outputs for the Same Input Across Different Models: GPT-4, CodeT5+, and CodeGen2.0.

After using the same prompt with CodeT5+ and CodeGen2.0, we found that neither model provided a reasonable answer based on the bug-fixing commit. For instance, when querying CodeT5+ about Cross-File Detections, where the bug-inducing commit is not in the same file as the bug-fixing commit, we sought to identify which file in the Linux kernel might be responsible. Unlike GPT-4, which typically suggests potential filenames or directories, CodeT5+ and CodeGen2.0 returned a nonsensical sentence. For example, as shown in Fig. 8, all

```

1655 - static int hdmi_lpe_audio_probe(struct platform_device *pdev)
1655 + static int __hdmi_lpe_audio_probe(struct platform_device *pdev)
...
1818 + static int hdmi_lpe_audio_probe(struct platform_device *pdev)
1819 + {
1820 +     return snd_card_free_on_error(&pdev->dev, __hdmi_lpe_audio_probe(pdev));
1821 + }

```

Fig. 9. Example of Failure Reason: Mapping Ghost.

three models were prompted to identify which Linux kernel file might be the cause of the bug-fixing commit 1e45d043 (as illustrated in Fig. 8(a)). GPT-4 correctly identified the potential directories related to the bug-inducing commit, and its response is displayed in Fig. 8(b). In contrast, CodeT5+ repeated responded with “#PF error: [normal kernel read fault]” (Fig. 8(c)), a sentence from the bug-fixing commit itself, offering no insight into the filename. CodeGen2.0’s response was also off target, returning unrelated license content, as detailed in Fig. 8(d). Similarly, with Function-Blame Detections, when attempting to use prompts to have CodeT5+ or CodeGen2.0 pinpoint the problematic line, both failed to specify a line, instead generating an extensive amount of code.

Our results demonstrate a significant limitation in the capabilities of current models, such as CodeT5+ and CodeGen, to offer precise, context-relevant assistance in identifying bug-inducing commits, particularly in complex scenarios involving cross-file analysis and specific line identification within large codebases like the Linux kernel. It also motivates the need for developing more sophisticated LLMs, incorporating instruction tuning and domain knowledge, to improve performance in detecting bug-inducing commits, possibly even surpassing GPT-4’s capabilities.

VIII. DISCUSSION

Although we categorized the failures of SZZ algorithms based on their location (i.e., function-log, within-file, or cross-file detection) in Section VI-C, we did not explore the reasons behind these failures, which could lead to improvements in the SZZ algorithm. In this section, we give a brief discussion based on our qualitative analysis.

(1) **Mapping Ghost:** This category of cases are similar to the Remove Mapping Ghost situation, where the lines removed in the bug-fixing commit do not directly assist in identifying the bug-inducing commit. However, they are crucial for resolving the bug. For instance, as shown in Fig. 9, in the bug-fixing commit 5e154df²⁵, the fix creates a wrapper that causes the original function to be called via a function pointer (line 1818-1821). Thus, it is necessary to change the name of the original function (line 1655), and this is an integral part of the bug fix. However, the removed line in line 1655, while crucial for the bug-fix, does not assist in identifying the bug-inducing commit. The core issue lies in the absence of a functionality rather than a defect in the existing functionality.

²⁵<https://github.com/torvalds/linux/commit/5e154dfb4f9995096aa6d342df75040ae802c17e>

(2) **Partial Bug-fixing:** This case occurs when a bug-fixing commit rectifies a previous, incorrect bug-fix attempt, and the bug-inducing commit, as labeled by the developer, is the one that the initial, flawed bug-fix was attempting to correct. The SZZ algorithm identifies the recent, incorrect fix as the bug-inducing commit. For example, the bug-fixing commit c0702a4²⁶ addressed a bug originally introduced by commit 297ccce. Prior to this, commit 8299699²⁷ attempted to fix the bug originating from commit 297ccce but failed, leading to the necessity of bug-fixing commit c0702a4.

(3) **Affected by Other Changes:** In some cases, code that was previously correct becomes problematic due to subsequent changes. These subsequent changes are then considered the bug-inducing commits. However, the SZZ algorithm might incorrectly identify the commit that originally wrote the code as the bug-inducing commit. For instance, the bug-fixing commit 9b7becf1²⁸ addresses a bug induced by a previous commit. The actual bug-inducing commit, 219fb0c, modified a function but failed to update the relevant failure message. While the change code made in commit 9b7becf1 was appropriate at the time, it became flawed when commit 219fb0c did not make the necessary update. However, the SZZ algorithm incorrectly identifies the commit that initially introduced the changed code as the bug-inducing one.

(4) **Developer Error:** Even developers are not infallible and can sometimes incorrectly label commits. For example, in the bug-fixing commit e04e7a7,²⁹ the developer identified commit 8195b13 as the bug-inducing commit. However, these two commits address different aspects of deadlock issues in hv_netvsc. The bug-fixing commit e04e7a7 resolves a deadlock caused by a race condition between two functions and their locking mechanisms. In contrast, the commit labeled by the developer as bug-inducing, 8195b13, deals with a deadlock resulting channel offer messages in the same queue. Moreover, after our review, the commit 7bf7bb3,³⁰ identified by the SZZ algorithm as the bug-inducing one, is considered correct. The rationale is that in commit 7bf7bb3, `rtnl_lock()` was added at an inappropriate location, leading to a deadlock. This can be understood from the code comments in the bug-fixing commit, as shown in Fig. 10. This revision emphasizes the role of `rtnl_lock()` in preventing the deadlock and clarifies why the SZZ algorithm's identification of the bug-inducing commit is considered accurate in this context.

IX. THREATS TO VALIDITY

Construct validity. Our dataset, collected from the Linux kernel serves as an excellent study subject. As outlined in Section III, we employed an automated approach to collect the data.

²⁶<https://github.com/torvalds/linux/commit/c0702a4bd41829f05638ec2dab70f6bb8d8010ce>

²⁷<https://github.com/torvalds/linux/commit/829969956f97e880dd01086be47747226e48a3f0>

²⁸<https://github.com/torvalds/linux/commit/9b7becf103e2689d7f005895130ccf89a153fef1>

²⁹<https://github.com/torvalds/linux/commit/e04e7a7bbd4bbabef4e1a58367e5fc9b2edc3b10>

³⁰<https://github.com/torvalds/linux/commit/7bf7bb37f16a80465ee3bd7c6c966f96f5a075a6>

```
2209 + /* We must get rtnl lock before scheduling nvdev->subchan_work,
2210 + * otherwise netvsc_subchan_work() can get rtnl lock first and wait...
2211 + * all subchannels to show up, but that may not happen because
2212 + * netvsc_probe() can't get rtnl lock and as a result vmbus_onoffer()
2213 + * -> ... -> device_add() -> ... -> __device_attach() can't get
2214 + * the device lock, so all the subchannels can't be processed -
2215 + * finally netvsc_subchan_work() hangs for ever.
2216 + */
2217 + rtnl_lock();
...
2227 - rtnl_lock();
```

Fig. 10. Example of Failure Reason: Developer Error.

By following the official Linux documentation for identifying bug-fixing commits, we ensured 100% accuracy in our collected data. A few bug-fixing commits may have been missed due to formatting errors, but this small percentage should have no impact on our results. We used a minimum number (the commit ID must have at least 7 characters) when searching for bug-inducing commits to minimize instances where a partial commit ID matches multiple bug-inducing commits.

As Linux Kernel does not have an issue tracker, e.g., Bugzilla or JIRA, it is challenging to ensure that all bugs are considered. Furthermore, the Linux documentation does not enforce the inclusion of bug-inducing commits when submitting bug-fixing commits, as the guideline suggests “If the patch fixes a bug, cite the commit which introduced the bug if possible (and please provide both the commit ID and the title when citing commits).” Therefore, it is possible that some bug-fixing commits may not be linked to their corresponding bug-inducing commits, depending on the developer’s discretion. Additionally, our dataset does not include extrinsic bugs as described by Rodriguez et al. [61], which are bug fixes not directly traceable to changes within the repository. As a result, our research does not claim to document every single bug-fixing commit from the past decade. Instead, our objective is to collate as many bug-fixing commits as possible, each with a verified origin, to offer a thorough and accurate overview.

Another threat to construct validity of our dataset is that around 96% of bug-fixing commits are linked to a single bug-inducing commit. This raises questions about the complexity of bugs. The majority of bug-fixing commits linked to a single bug-inducing commit may be indicative of developers conforming to specific guidelines within the Linux kernel project, potentially introducing a bias into the dataset. Our comparison with prior research by Rosa et al. [28] revealed a similar trend: out of 1,115 bug-fixing commits, 1,113 were associated with just one bug-inducing commit. Given these findings, it becomes challenging to definitively assert whether the reality of bug genesis is more accurately reflected in bug-fixing commits with a single or multiple bug-inducing commits. Therefore, we leave this as an open question, acknowledging the need for further discussion and exploration in this area.

In our study, we conducted the evaluation of SZZ implementations solely at the commit level, aligning with the methodology used by Rosa et al. [28]. It is important to note that the

Linux kernel does not record pull requests (PR). Consequently, our evaluation did not extend to the PR level.

Internal validity. There may be some pitfalls in automatically collecting bug-inducing commits. After selecting commits with “Fixes:”, 2,358 (3%) of the commits could not be matched to any bug-inducing commit. These commit IDs are stored in a dedicated dataset. We manually label the commit ids in this *abnormal dataset* and divide them into two categories: *partial commit ID* and *not in the repository*.

Partial commit ID indicates commits where the text after the keyword “Fixes:” is not recognized by the automated filtering technique as a commit ID. The filtering technique checks the commit ID using the regular expression `[0-9a-f]{7,40}`, requiring 7-40 characters. The total number of *Partial commit ID* commits is 1,021 (1.3%). They include four causes: (i) the content after Fixes is a link to a bug report; (e.g., Fixes: http://bugs.gentoo.org/show_bug.cgi?id=87182) (ii) the content after Fixes gives some description of the bug; (iii) the content after Fixes has some format problems (e.g., Fixes: commit fda789fda); (iv) the commit ID after Fixes: may be too short.

Not in the repository indicates that the method can recognize the commit ID in the commit message, however, the commit ID does not exist in the list of commit IDs fetched from the master branch of the Linux kernel repository. The number of commits in this situation is 1,337 (1.7%). It includes three prominent cases: (i) format problem: irregular formatting making it impossible to search for the commit ID (e.g., fda798fda:, 7598e8700e9a(drm/i915/gvt; (ii) rebase problem: the commit is not in any branch of the repository, and may belong to a fork outside of the repository; (iii) branch problem: we only focus on the master branch, while some commits may occur in another branch.

However, in total, anomalous commits only account for 3% of the total number of commits. We have made these abnormal commits publicly available in a CSV file for further study. So, we believe that the threat is minimal.

To minimize threats to internal validity from implementation errors, we use the implementation of MA-SZZ from the replication package of Rosa et al. [28] without making modifications. However, a deeper analysis found that the implementation of MA-SZZ by Rosa et al. [28] diverged from MA-SZZ’s original implementation [12]. The implementation of Rosa et al. included additional modifications for detecting the movement of code within files and from other files.

Another threat to internal validity is that some bug-fixing commits are also bug-inducing commits. In our analysis, we found that there are 4,517 such common commits in our dataset, which represent both bug-fixing and bug-inducing actions. These common commits account for only 5.9% out of the total 76,046 unique commits. To mitigate this threat, we have listed all these commits in a file named “common_hashes.txt” in the replication package. This collection of cases where developers introduce bugs while fixing bugs may provide opportunities for further study.

External validity. Considering that our study focuses on a single software project, primarily utilizing the C programming language, our findings may not generalize to other projects

or programming languages. However, the Linux kernel is a well-structured software repository with a substantial code size (Linux v6.0 contains 23 MLOC). Therefore, it serves as a valuable research object for evaluating the effectiveness of SZZ algorithms.

X. CONCLUSION AND FUTURE WORK

SZZ is a vital algorithm for mining software repositories, and many studies rely on its results. Ensuring the precision and recall of SZZ in real scenarios is thus crucial.

In our study, we collected a dataset of 76K bug-fixing commits linked to bug-inducing commits in the Linux kernel. We evaluated six SZZ algorithms on this dataset, analyzed the ghost commits issue, and examined its impact on SZZ. We proposed TC-SZZ, which can find a bug-inducing commit for 17.7% of the failure cases. Upon investigating the remaining failures, we found that in 37.9% of the cases, the bug-inducing commit is not in the file history. ChatGPT demonstrates potential in assisting the SZZ algorithm in identifying the associated bug-inducing commits in these challenging cases.

Our publicly available dataset of 76K bug-fixing commits with their bug-inducing commits provides a unique opportunity for advancing SZZ algorithm research, including training Large Language Models (LLMs) for improved precision and recall. Key areas for future exploration include: (1) **Enhancing SZZ Algorithm Precision:** Despite R-SZZ achieving 59% precision, there is significant potential for improvement. Future studies should explore patterns beyond non-semantic source code, meta-changes, and refactoring changes to increase accuracy. (2) **Refactoring Solutions Across Programming Languages:** The extension of refactoring solutions, typically focused on Java, to other programming languages like C in our Linux dataset, remains a challenge needing attention. (3) **Addressing Undefined Failures in Recall:** Investigating the 23.08% of previously undefined failure cases, alongside ‘Ghost Commit’ issues, is crucial for a more comprehensive understanding and improvement of recall. (4) **Impacts on Downstream Tasks:** Utilizing our dataset to evaluate the effect of SZZ algorithms on downstream tasks, such as Just-in-Time defect prediction, offers a promising direction for future research. These directions not only aim to refine SZZ algorithms but also seek to understand their broader implications in software engineering research.

DATA AVAILABILITY

The dataset used in this paper, the TC-SZZ implementation, and the results of ChatGPT experiments are publicly available at <https://doi.org/10.6084/m9.figshare.23889792.v2>.

ACKNOWLEDGMENT

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

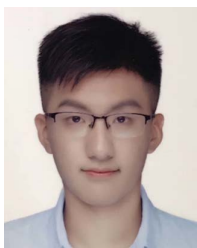
REFERENCES

- [1] J. Śliwinski, T. Zimmermann, and A. Zeller, “When do changes induce fixes?” in *Proc. Int. Workshop Mining Softw. Repositories, (MSR)*, 2005, pp. 1–5.

- [2] S. Kim, T. Zimmermann, K. Pan, and E. James Jr, "Automatic identification of bug-introducing changes," in *Proc. 21st IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE'06)*, Piscataway, NJ, USA: IEEE Press, 2006, pp. 81–90.
- [3] H. Hata, O. Mizuno, and T. Kikuno, "Bug prediction based on fine-grained module histories," in *Proc. 34th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2012, pp. 200–210.
- [4] M. Tan, L. Tan, S. Dara, and C. Mayeux, "Online defect prediction for imbalanced data," in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.*, vol. 2, Piscataway, NJ, USA: IEEE Press, 2015, pp. 99–108.
- [5] L. Pascarella, F. Palomba, and A. Bacchelli, "Fine-grained just-in-time defect prediction," *J. Syst. Softw.*, vol. 150, pp. 22–36, Apr. 2019.
- [6] M. Yan, X. Xia, Y. Fan, A. E. Hassan, D. Lo, and S. Li, "Just-in-time defect identification and localization: A two-phase framework," *IEEE Trans. Softw. Eng.*, vol. 48, no. 1, pp. 82–101, Jan. 2022.
- [7] Y. Fan, X. Xia, D. A. Da Costa, D. Lo, A. E. Hassan, and S. Li, "The impact of mislabeled changes by SZZ on just-in-time defect prediction," *IEEE Trans. Softw. Eng.*, vol. 47, no. 8, pp. 1559–1586, Aug. 2021.
- [8] G. Bavota and B. Russo, "Four eyes are better than two: On the impact of code reviews on software quality," in *Proc. IEEE Int. Conf. Softw. Maintenance Evolution (ICSME)*, Piscataway, NJ, USA: IEEE Press, 2015, pp. 81–90.
- [9] M. Tufano, G. Bavota, D. Poshyanyk, M. Di Penta, R. Oliveto, and A. De Lucia, "An empirical study on developer-related factors characterizing fix-inducing commits," *J. Softw.: Evol. Process*, vol. 29, no. 1, 2017, Art. no. e1797.
- [10] H. Aman, S. Amasaki, T. Yokogawa, and M. Kawahara, "Empirical study of fault introduction focusing on the similarity among local variable names," in *Proc. 7th Int. Workshop Quantitative Approaches Softw. Qual. (QuASoQ)*, 2019, pp. 3–11.
- [11] B. Chen and Z. M. J. Jiang, "Extracting and studying the logging-code-issue-introducing changes in Java-based large-scale open source software systems," *Empirical Softw. Eng.*, vol. 24, no. 4, pp. 2285–2322, 2019.
- [12] D. A. Da Costa, S. McIntosh, W. Shang, U. Kulesza, R. Coelho, and A. E. Hassan, "A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes," *IEEE Trans. Softw. Eng.*, vol. 43, no. 7, pp. 641–657, Jul. 2017.
- [13] E. C. Neto, D. A. Da Costa, and U. Kulesza, "The impact of refactoring changes on the SZZ algorithm: An empirical study," in *Proc. IEEE 25th Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 380–390.
- [14] E. C. Neto, D. A. da Costa, and U. Kulesza, "Revisiting and improving SZZ implementations," in *Proc. ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 1–12.
- [15] C. Williams and J. Spacco, "SZZ revisited: Verifying when changes induce fixes," in *Proc. Workshop Defects Large Softw. Syst.*, 2008, pp. 32–36.
- [16] S. Davies, M. Roper, and M. Wood, "Comparing text-based and dependence-based approaches for determining the origins of bugs," *J. Softw.: Evol. Process*, vol. 26, no. 1, pp. 107–139, 2014.
- [17] C. Rezk, Y. Kamei, and S. McIntosh, "The ghost commit problem when identifying fix-inducing changes: An empirical study of apache projects," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3297–3309, Sep. 2022.
- [18] K. Herzig and A. Zeller, "The impact of tangled code changes," in *Proc. 10th Work. Conf. Mining Softw. Repositories (MSR)*, Piscataway, NJ, USA: IEEE Press, 2013, pp. 121–130.
- [19] E. Sahal and A. Tosun, "Identifying bug-inducing changes for code additions," in *Proc. 12th ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas.*, 2018, pp. 1–2.
- [20] S. Kim, E. J. Whitehead, and Y. Zhang, "Classifying software changes: Clean or buggy?" *IEEE Trans. Softw. Eng.*, vol. 34, no. 2, pp. 181–196, Mar./Apr. 2008.
- [21] Z. Tóth, P. Gyimesi, and R. Ferenc, "A public bug database of GitHub projects and its application in bug prediction," in *Proc. Int. Conf. Comput. Sci. Appl.*, Cham, Switzerland: Springer, 2016, pp. 625–638.
- [22] M. Wen, R. Wu, and S.-C. Cheung, "Locus: Locating bugs from software changes," in *Proc. 31st IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2016, pp. 262–273.
- [23] S. Kim, T. Zimmermann, E. J. Whitehead Jr, and A. Zeller, "Predicting faults from cached history," in *Proc. 29th Int. Conf. Softw. Eng. (ICSE'07)*, Piscataway, NJ, USA: IEEE Press, 2007, pp. 489–498.
- [24] F. Rahman, D. Posnett, A. Hindle, E. Barr, and P. Devanbu, "BugCache for inspections: Hit or miss?" in *Proc. 19th ACM SIGSOFT Symp. 13th Eur. Conf. Found. Softw. Eng.*, 2011, pp. 322–331.
- [25] J. Eyolfson, L. Tan, and P. Lam, "Correlations between bugginess and time-based commit characteristics," *Empirical Softw. Eng.*, vol. 19, no. 4, pp. 1009–1039, 2014.
- [26] M. L. Bernardi, G. Canfora, G. A. Di Lucca, M. Di Penta, and D. Distant, "The relation between developers' communication and fix-inducing changes: An empirical study," *J. Syst. Softw.*, vol. 140, pp. 111–125, Jun. 2018.
- [27] D. Izquierdo-Cortazar, G. Robles, and J. M. González-Barahona, "Do more experienced developers introduce fewer bugs?" in *Proc. IFIP Int. Conf. Open Source Syst.*, Berlin, Heidelberg: Springer, 2012, pp. 268–273.
- [28] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, and R. Oliveto, "Evaluating SZZ implementations through a developer-informed oracle," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 436–447.
- [29] D. Silva and M. T. Valente, "RefDiff: Detecting refactorings in version histories," in *Proc. IEEE/ACM 14th Int. Conf. Mining Softw. Repositories (MSR)*, Piscataway, NJ, USA: IEEE Press, 2017, pp. 269–279.
- [30] N. Tsantalis, M. Mansouri, L. M. Eshkevari, D. Mazinanian, and D. Dig, "Accurate and efficient refactoring detection in commit history," in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 483–494.
- [31] Q. Zhong, L. Ding, J. Liu, B. Du, and D. Tao, "Can ChatGPT understand too? A comparative study on ChatGPT and fine-tuned BERT," 2023, *arXiv:2302.10198*.
- [32] J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, "ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design," 2023, *arXiv:2303.07839*.
- [33] D. Spadini, M. Niche, and A. Bacchelli, "PyDriller: Python framework for mining software repositories," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2018, pp. 908–911.
- [34] M. Borg, O. Svensson, K. Berg, and D. Hansson, "SZZ unleashed: An open implementation of the SZZ algorithm-featuring example usage in a study of just-in-time bug prediction for the Jenkins project," in *Proc. 3rd ACM SIGSOFT Int. Workshop Mach. Learn. Techn. Softw. Qual. Eval.*, 2019, pp. 7–12.
- [35] V. Lenarduzzi, F. Palomba, D. Taibi, and D. A. Tamburri, "Open-SZZ: A free, open-source, web-accessible implementation of the SZZ algorithm," in *Proc. 28th Int. Conf. Program Comprehension*, 2020, pp. 446–450.
- [36] S. Herbold, A. Trautsch, F. Trautsch, and B. Ledel, "Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection," *Empirical Softw. Eng.*, vol. 27, no. 2, pp. 1–49, 2022.
- [37] K. Herzig, S. Just, and A. Zeller, "It's not a bug, it's a feature: How misclassification impacts bug prediction," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2013, pp. 392–401.
- [38] C. Bird, A. Bachmann, E. Aune, J. Duffy, A. Bernstein, V. Filkov, and P. Devanbu, "Fair and balanced? Bias in bug-fix datasets," in *Proc. 7th Joint Meeting Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Found. Softw. Eng.*, 2009, pp. 121–130.
- [39] T. H. Nguyen, B. Adams, and A. E. Hassan, "A case study of bias in bug-fix datasets," in *Proc. 17th Work. Conf. Reverse Eng.*, Piscataway, NJ, USA: IEEE Press, 2010, pp. 259–268.
- [40] R. Wu, H. Zhang, S. Kim, and S.-C. Cheung, "ReLink: Recovering links between bugs and changes," in *Proc. 19th ACM SIGSOFT Symp. 13th Eur. Conf. Found. Softw. Eng.*, 2011, pp. 15–25.
- [41] S. McIntosh and Y. Kamei, "Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction," in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 560–560.
- [42] T. Zimmermann, S. Kim, A. Zeller, and E. J. Whitehead Jr, "Mining version archives for co-changed lines," in *Proc. Int. Workshop Mining Softw. Repositories*, 2006, pp. 72–75.
- [43] C. C. Williams and J. W. Spacco, "Branching and merging in the repository," in *Proc. Int. Work. Conf. Mining Softw. Repositories*, 2008, pp. 19–22.
- [44] J. Pace, "A tool which compares Java files based on content," Jpace. Accessed: Jan. 15, 2024. [Online]. Available: <http://www.incava.org/projects/java/diff>
- [45] G. Rodríguez-Pérez, G. Robles, and J. M. González-Barahona, "Reproducibility and credibility in empirical software engineering: A case study based on a systematic literature review of the use of the SZZ algorithm," *Inf. Softw. Technol.*, vol. 99, pp. 164–176, Jul. 2018.
- [46] F. Petrulio et al., "SZZ in the time of pull requests," 2022, *arXiv:2209.03311*.
- [47] P. Bludau and A. Pretschner, "PR-SZZ: How pull requests can support the tracing of defects in software repositories," in *Proc. IEEE Int. Conf.*

Softw. Anal., Evol. Reeng. (SANER), Piscataway, NJ, USA: IEEE Press, 2022, pp. 1–12.

- [48] L. Bao, X. Xia, A. E. Hassan, and X. Yang, “V-SZZ: Automatic identification of version ranges affected by CVE vulnerabilities,” in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 2352–2364.
- [49] L. Erlikh, “Leveraging legacy system dollars for e-business,” *IT Professional*, vol. 2, no. 3, pp. 17–23, 2000.
- [50] R. Minelli, A. Mocci, and M. Lanza, “I know what you did last summer—an investigation of how developers spend their time,” in *Proc. IEEE 23rd Int. Conf. Program Comprehension*, Piscataway, NJ, USA: IEEE Press, 2015, pp. 25–35.
- [51] R. Baeza-Yates et al., *Modern Information Retrieval*, vol. 463. New York, NY, USA: ACM, 1999.
- [52] C. D. Manning, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.
- [53] R. Oliveto, M. Gethers, D. Poshvanyk, and A. De Lucia, “On the equivalence of information retrieval methods for automated traceability link recovery,” in *Proc. IEEE 18th Int. Conf. Program Comprehension*, Piscataway, NJ, USA: IEEE Press, 2010, pp. 68–71.
- [54] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*. Evanston, IL, USA: Routledge, 2013.
- [55] H. Hata, C. Treude, R. G. Kula, and T. Ishio, “9.6 million links in source code comments: Purpose, evolution, and decay,” in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 1211–1221.
- [56] J. E. Montandon, C. Politowski, L. L. Silva, M. T. Valente, F. Petrillo, and Y.-G. Guéhéneuc, “What skills do it companies look for in new developers? A study with stack overflow jobs,” *Inf. Softw. Technol.*, vol. 129, 2021, Art. no. 106429.
- [57] M. Aniche et al., “How modern news aggregators help development communities shape and share knowledge,” in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 499–510.
- [58] J. Wang, L. Li, and A. Zeller, “Restoring execution environments of Jupyter notebooks,” in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1622–1633.
- [59] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, “CodeT5+: Open code large language models for code understanding and generation,” 2023, *arXiv:2305.07922*.
- [60] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, “CodeGen2: Lessons for training LLMs on programming and natural languages,” 2023, *arXiv:2305.02309*.
- [61] G. Rodriguez-Perez, M. Nagappan, and G. Robles, “Watch out for extrinsic bugs! A case study of their impact in just-in-time bug prediction models on the OpenStack project,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 4, pp. 1400–1416, Apr. 2022.



Yunbo Lyu is a Ph.D. student and a Research Engineer with the School of Computing and Information Systems, Singapore Management University under the supervision of Prof. David Lo. His research interests include mining software repositories and library usage. His work has been published in high-quality software engineering conferences such as ICSE. For more information, see <https://yunbolyu.github.io>.



Hong Jin Kang received the Ph.D. degree from Singapore Management University. He is an Incoming Faculty Member with the University of Sydney. He is currently a Postdoctoral Fellow with the University of California, Los Angeles. His research interest includes aims to improve developer productivity by leveraging human knowledge through methods such as active learning. For more information, see <https://kanghj.github.io/>.



Ratnadira Widyasari is a Ph.D. candidate with the School of Computing and Information Systems, Singapore Management University, under the supervision of Prof. David Lo. Her research interests include the topic of artificial intelligence and explainability for software engineering, specifically for software quality assurance. For more information, see <https://ratnadiraw.github.io/>.



Julia Lawall received the Ph.D. degree from Indiana University, in 1994. She is a Researcher with Inria Paris, where she is the Head of the Whisper team. Her research interests include systems and software engineering. Since 2004, she has been designing and maintaining the program transformation tool Coccinelle, which has been extensively used on the Linux kernel.



David Lo (Fellow, IEEE) is the OUB Chair Professor of Computer Science and the Director of the Center for Research in Intelligent Software Engineering (RISE) with Singapore Management University. Championing the area of AI for Software Engineering (AI4SE) since the mid-2000s, he has demonstrated how AI—encompassing data mining, machine learning, information retrieval, natural language processing, and search-based algorithms—can transform software engineering data into automation and insights. His contributions have led

to over 20 awards including two Test-of-Time Awards and ten ACM SIGSOFT/IEEE TCSE Distinguished Paper Awards and gathered more than 30k citations. He is an ACM Fellow, ASE Fellow, and National Research Foundation Investigator (Senior Fellow), he has also served as a PC Co-Chair for ASE’20, FSE’24, and ICSE’25. For more information, see <http://www.mysmu.edu/faculty/davidlo/>.